PRACTICAL :-1

AIM: Implementation of Perceptron Learning Algorithm for Admission Prediction.

Problem Statement:-

Design and implement a single-layer Perceptron Neural Network to classify student admission eligibility using entrance examination marks and interview scores. Train the model on linearly separable data, predict admission outcomes for new applicants, visualize the decision boundary, and evaluate the impact of learning rate and epochs on the training process.

Source Code:-

```
import numpy as np
import matplotlib.pyplot as plt

# Admission Dataset
X = np.array([
    [35, 40],
    [40, 45],
    [45, 48],
    [50, 52],
    [55, 58],
    [70, 72],
    [75, 80],
    [82, 85],
    [90, 92]
], dtype=float)

# Feature Scaling
X = X / 100

y = np.array([0, 0, 0, 0, 0, 1, 1, 1, 1])


# Perceptron Class
class Perceptron:

    def __init__(self, learning_rate=0.1, epochs=50):
        self.lr = learning_rate
        self.epochs = epochs

    def train(self, X, y):
        self.weights = np.zeros(X.shape[1])
```

```
        self.bias = 0
        self.errors = []

        for epoch in range(self.epochs):
            error_count = 0

            for xi, target in zip(X, y):
                activation = np.dot(xi, self.weights) + self.bias
                prediction = 1 if activation >= 0 else 0

                error = target - prediction

                self.weights += self.lr * error * xi
                self.bias += self.lr * error

                if error != 0:
                    error_count += 1

            self.errors.append(error_count)
            print(f"Epoch {epoch+1:2d} Errors = {error_count}")

            if error_count == 0:
                print("\nTraining Converged.")
                break

    def predict(self, X):
        activation = np.dot(X, self.weights) + self.bias
        return np.where(activation >= 0, 1, 0)


# Create Model
model = Perceptron()
model.train(X, y)

# Accuracy
pred = model.predict(X)
accuracy = np.mean(pred == y) * 100

print("\nAccuracy =", accuracy, "%")


# Test New Student
print("\nAdmission Prediction")
```

```
entrance = float(input("Entrance Marks : ")) / 100
interview = float(input("Interview Score : ")) / 100

new = np.array([[entrance, interview]])

result = model.predict(new)

if result[0] == 1:
    print("\nEligible for Admission")
else:
    print("\nNot Eligible for Admission")


# Plot Dataset
plt.figure(figsize=(7, 6))

for i in range(len(X)):
    if y[i] == 0:
        plt.scatter(
            X[i, 0],
             X[i, 1],
            color="red",
            label="Not Admitted" if i == 0 else ""
        )
    else:
        plt.scatter(
            X[i, 0],
             X[i, 1],
            color="blue",
            label="Admitted" if i == 5 else ""
        )

x = np.linspace(0.3, 1, 100)
y_line = -(model.weights[0] * x + model.bias) / model.weights[1]

plt.plot(x, y_line, color="green", label="Decision Boundary")

plt.xlabel("Entrance Marks (Normalized)")
plt.ylabel("Interview Score (Normalized)")
plt.title("Admission Prediction using Perceptron")

plt.legend()
plt.grid(True)
plt.show()
```

```
# Learning Curve
plt.figure(figsize=(6, 4))

plt.plot(
    range(1, len(model.errors) + 1),
    model.errors,
    marker="o"
)

plt.xlabel("Epoch")
plt.ylabel("Classification Errors")
plt.title("Learning Curve")

plt.grid(True)
plt.show()
```

OUTPUT:-

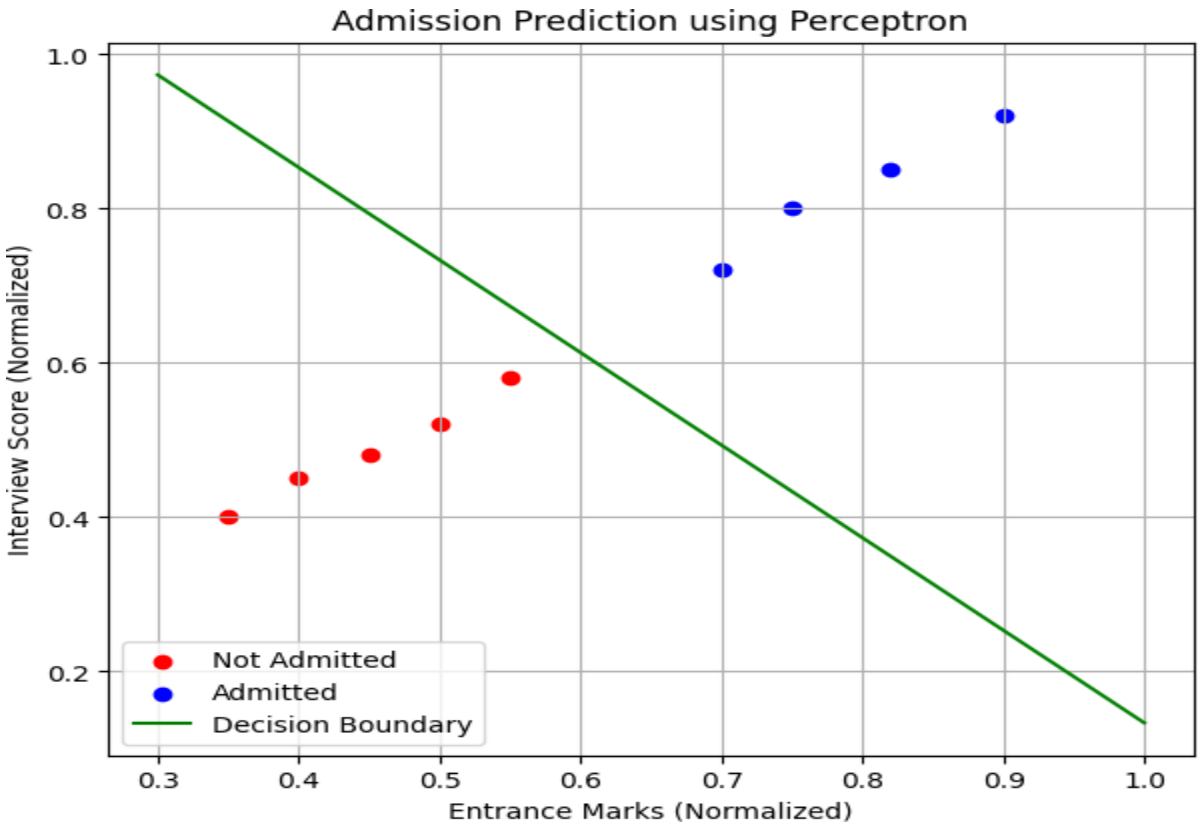
```
Epoch  1 Errors = 2
Epoch  2 Errors = 2
Epoch  3 Errors = 2
Epoch  4 Errors = 2
Epoch  5 Errors = 3
Epoch  6 Errors = 2
Epoch  7 Errors = 2
Epoch  8 Errors = 2
Epoch  9 Errors = 1
Epoch 10 Errors = 0

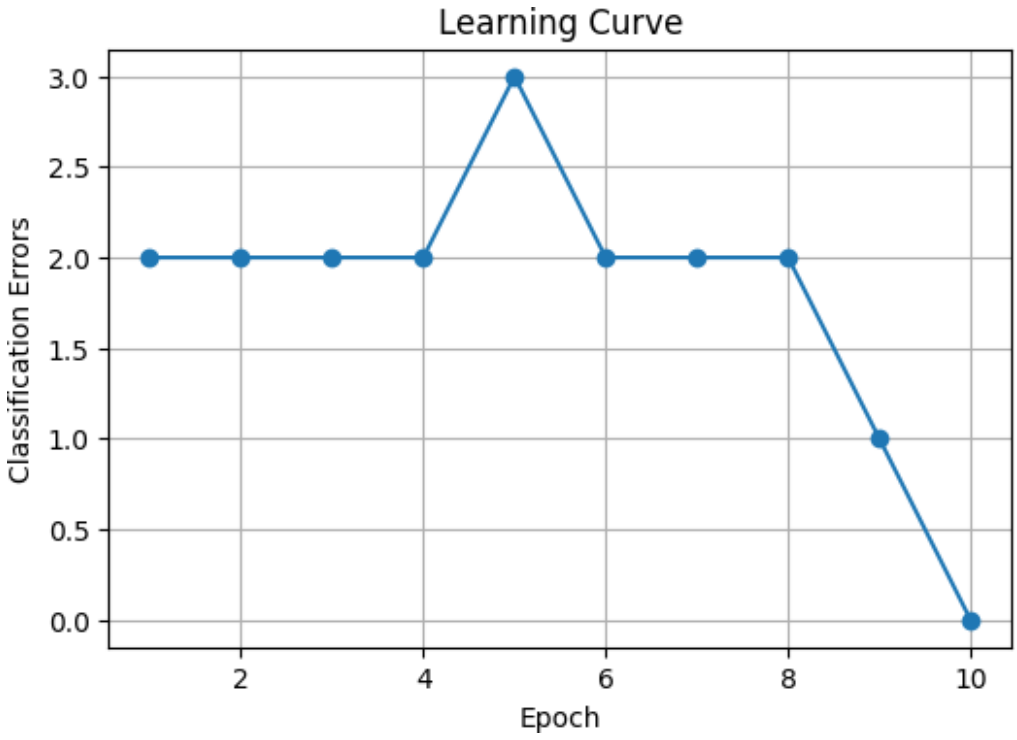
Training Converged.

Accuracy = 100.0 %

Admission Prediction
Entrance Marks : 50
Interview Score : 60

Not Eligible for Admission
```


Admission Prediction using Perceptron
Interview Score (Normalized)
Entrance Marks (Normalized)
Not Admitted
Admitted
Decision Boundary
1.0
0.8
0.6
0.4
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0


Learning Curve
Classification Errors
Epoch
3.0
2.5
2.0
1.5
1.0
0.5
0.0
2
4
6
8
10

Assignment:01:-

Implementation of Perceptron Learning Algorithm for Student Placement Prediction .

Problem Statement :-

Develop a single-layer Perceptron Neural Network to predict whether a student is likely to be Placed or Not Placed based on academic performance indicators such as CGPA and coding assessment score. Train the model using supervised learning, classify new student records, visualize the decision boundary, and analyze the effect of learning rate and epochs on model convergence and classification accuracy.

CODE:-

```
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------
# Dataset
# CGPA , Coding Score
# ------------------------------
 X = np.array([
    [5.5, 40],
    [6.0, 45],
    [6.2, 50],
    [6.5, 55],
    [6.8, 58],
    [7.2, 65],
    [7.5, 70],
    [8.0, 75],
    [8.3, 80],
    [8.5, 85],
    [8.8, 90],
    [9.2, 95]
], dtype=float)

# Output
# 0 = Not Placed
# 1 = Placed

y = np.array([0,0,0,0,0,1,1,1,1,1,1,1])

# ------------------------------
# Feature Scaling
```

```
# ------------------------------
X[:,0] = X[:,0] / 10
X[:,1] = X[:,1] / 100

# ------------------------------
# Perceptron Class
# ------------------------------

class Perceptron:

    def __init__(self, learning_rate=0.1, epochs=50):
        self.lr = learning_rate
        self.epochs = epochs

    def train(self, X, y):

        self.weights = np.zeros(X.shape[1])
        self.bias = 0
        self.errors = []

        for epoch in range(self.epochs):

            error_count = 0

            for xi, target in zip(X, y):

                activation = np.dot(xi, self.weights) + self.bias

                prediction = 1 if activation >= 0 else 0

                error = target - prediction

                self.weights += self.lr * error * xi
                self.bias += self.lr * error

                if error != 0:
                    error_count += 1

             self.errors.append(error_count)

            print("Epoch", epoch+1, "Errors =", error_count)

            if error_count == 0:
                print("\nTraining Converged")
```

```
                break

    def predict(self, X):

        activation = np.dot(X, self.weights) + self.bias

        return np.where(activation >= 0, 1, 0)


# ------------------------------
# Train Model
# ------------------------------

model = Perceptron(learning_rate=0.1, epochs=50)

model.train(X, y)

# ------------------------------
# Accuracy
# ------------------------------

prediction = model.predict(X)

accuracy = np.mean(prediction == y) * 100

print("\nAccuracy =", accuracy,"%")

# ------------------------------
# Test New Student
# ------------------------------

print("\nPlacement Prediction")

cgpa = float(input("Enter CGPA (Out of 10): "))
coding = float(input("Enter Coding Score (Out of 100): "))

new_student = np.array([[cgpa/10, coding/100]])

result = model.predict(new_student)

if result[0] == 1:
    print("\nStudent is Likely to be PLACED")
else:
    print("\nStudent is NOT LIKELY to be PLACED")
```

```
# ------------------------------
# Plot Dataset
# ------------------------------

plt.figure(figsize=(7,6))

for i in range(len(X)):

    if y[i] == 0:
        plt.scatter(
            X[i,0],
            X[i,1],
            color='red',
            s=100,
            label='Not Placed' if i==0 else ""
        )

    else:
        plt.scatter(
            X[i,0],
            X[i,1],
            color='blue',
            s=100,
            label='Placed' if i==5 else ""
        )

x = np.linspace(0.4,1.0,100)

y_line = -(model.weights[0]*x + model.bias)/model.weights[1]

plt.plot(x, y_line, color='green', linewidth=2, label='Decision Boundary')

plt.xlabel("CGPA (Normalized)")
plt.ylabel("Coding Score (Normalized)")
plt.title("Student Placement Prediction using Perceptron")
plt.legend()
plt.grid(True)
plt.show()

# ------------------------------
# Learning Curve
# ------------------------------
```

```
plt.figure(figsize=(6,4))

plt.plot(
    range(1, len(model.errors)+1),
    model.errors,
    marker='o',
    color='purple'
)

plt.xlabel("Epoch")
plt.ylabel("Classification Errors")
plt.title("Learning Curve")
plt.grid(True)
plt.show()
```

OUTPUT:-

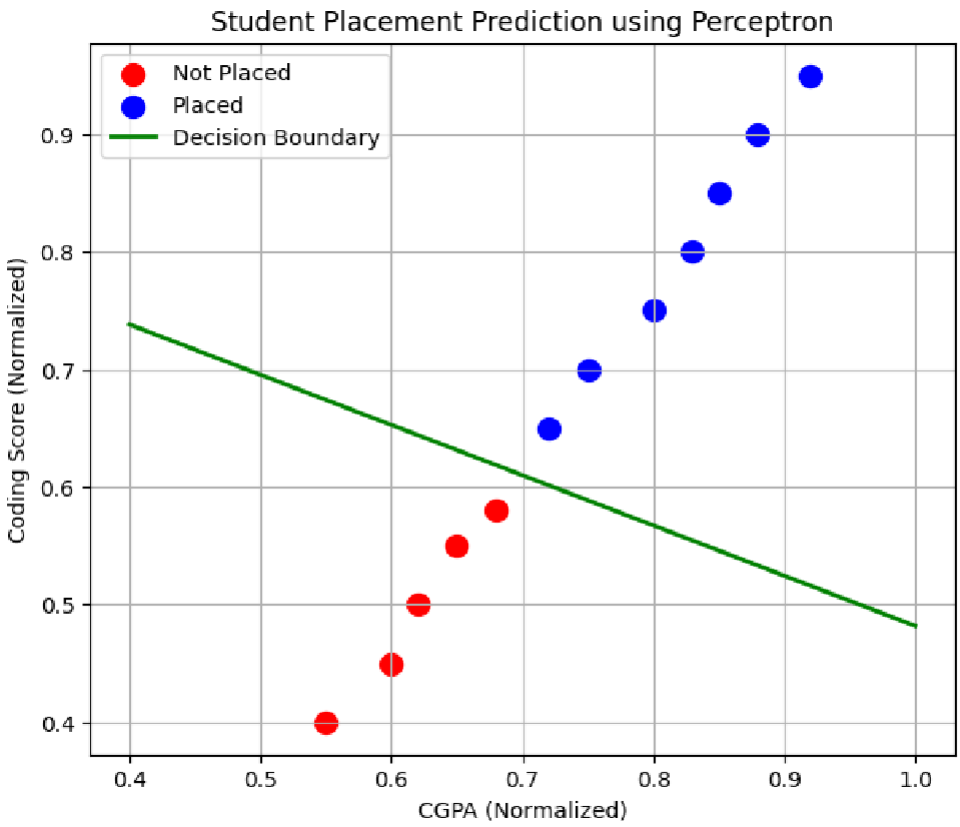
```
Epoch 1 Errors = 2
Epoch 2 Errors = 2
Epoch 3 Errors = 2
Epoch 4 Errors = 2
Epoch 5 Errors = 2
Epoch 6 Errors = 2
Epoch 7 Errors = 1
Epoch 8 Errors = 0

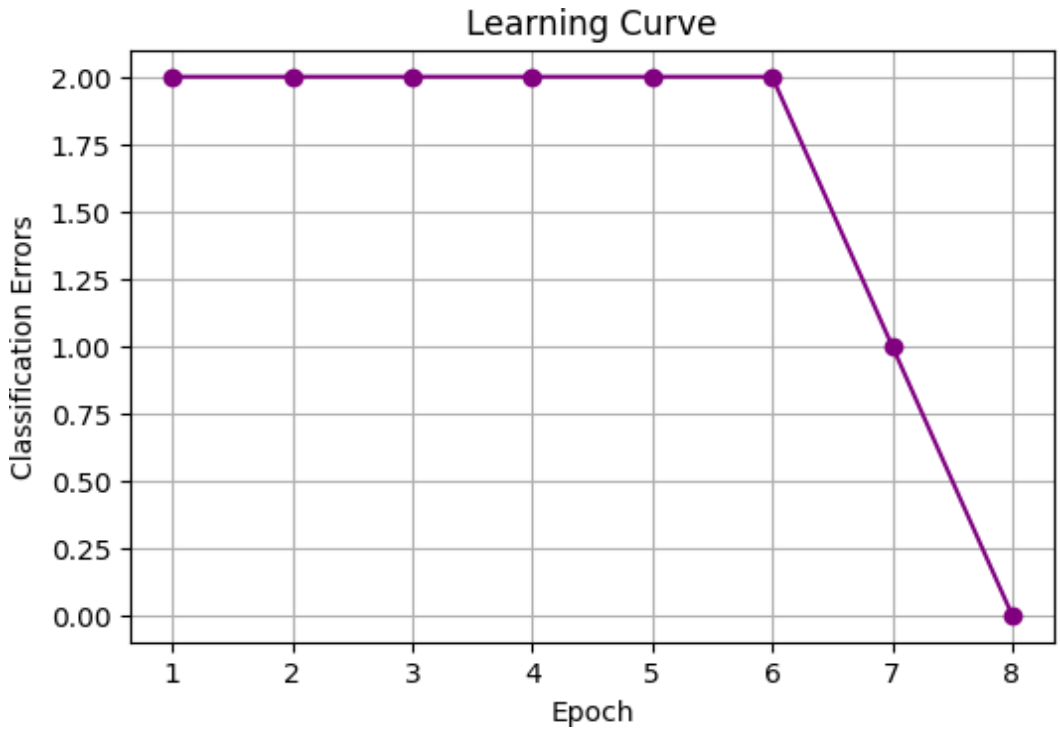
Training Converged

Accuracy = 100.0 %

Placement Prediction
Enter CGPA (Out of 10): 7
Enter Coding Score (Out of 100): 85

Student is Likely to be PLACED
```


Student Placement Prediction using Perceptron
Not Placed
Placed
Decision Boundary
Coding Score (Normalized)
CGPA (Normalized)
0.4
0.5
0.6
0.7
0.8
0.9
1.0


Learning Curve
Classification Errors
Epoch
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
1
2
3
4
5
6
7
8

Assignment:02:-

Loan Approval Prediction using Perceptron Learning Algorithm

Problem Statement:-

Develop a single-layer Perceptron Neural Network to classify loan applications as Approved or Rejected based on applicant attributes such as annual income and credit score. Train the model using labeled data, visualize the decision boundary, and investigate the effect of learning rate and epochs on classification performance.

CODE:-

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Annual_Income": [
        25000, 30000, 35000, 40000, 42000,
        45000, 48000, 50000, 52000, 55000,
        58000, 60000, 62000, 65000, 68000,
        70000, 75000, 80000, 85000, 90000
    ],
    "Credit_Score": [
        550, 580, 600, 610, 620,
        630, 640, 650, 660, 670,
        680, 690, 700, 710, 720,
        730, 740, 750, 760, 780
    ],
    "Loan_Status": [
        0, 0, 0, 0, 0,
        0, 0, 1, 1, 1,
        1, 1, 1, 1, 1,
        1, 1, 1, 1, 1
    ]
}

df = pd.DataFrame(data)
```

```
print("=" * 60)
print("LOAN APPROVAL DATASET")
print("=" * 60)
print(df.to_string(index=False))

X = df[["Annual_Income", "Credit_Score"]].values
y = df["Loan_Status"].values

X_mean = X.mean(axis=0)
X_std = X.std(axis=0)
X_scaled = (X - X_mean) / X_std


class Perceptron:

    def _init_(self, learning_rate=0.01, epochs=100):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = 0
        self.errors = []

    def activation(self, value):
        if value >= 0:
            return 1
        else:
            return 0

    def predict(self, X):
        linear_output = np.dot(X, self.weights) + self.bias
        return np.array([
            self.activation(value)
            for value in linear_output
        ])

    def fit(self, X, y):
        self.weights = np.zeros(X.shape[1])
        self.bias = 0
```

```
        self.errors = []

        for epoch in range(self.epochs):
            total_errors = 0

            for xi, target in zip(X, y):
                prediction = self.activation(
                    np.dot(xi, self.weights) + self.bias
                )

                update = self.learning_rate * (target - prediction)

                self.weights += update * xi
                self.bias += update

                if update != 0:
                    total_errors += 1

            self.errors.append(total_errors)

            if total_errors == 0:
                break

        return self


learning_rate = 0.01
epochs = 100

model = Perceptron(
    learning_rate=learning_rate,
    epochs=epochs
)

model.fit(X_scaled, y)

predictions = model.predict(X_scaled)
```

```
df["Predicted_Status"] = predictions

df["Prediction_Result"] = np.where(
    predictions == df["Loan_Status"],
    "Correct",
    "Incorrect"
)

accuracy = np.mean(predictions == y) * 100

print("\n" + "=" * 60)
print("MODEL RESULTS")
print("=" * 60)

print("Learning Rate :", learning_rate)
print("Maximum Epochs:", epochs)
print("Actual Epochs :", len(model.errors))

print("\nFinal Weights:")
print(model.weights)

print("\nBias:")
print(model.bias)

print("\nAccuracy: {:.2f}%".format(accuracy))

print("\n" + "=" * 60)
print("PREDICTIONS")
print("=" * 60)

print(
    df[
        [
            "Annual_Income",
            "Credit_Score",
            "Loan_Status",
            "Predicted_Status",
            "Prediction_Result"
```

```
        ]
    ].to_string(index=False)
)

plt.figure(figsize=(8, 5))

plt.plot(
    range(1, len(model.errors) + 1),
    model.errors,
    marker="o",
    color="blue"
)

plt.xlabel("Epoch")
plt.ylabel("Number of Misclassifications")
plt.title("Perceptron Training Error")
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 6))

plt.scatter(
    X_scaled[y == 0, 0],
    X_scaled[y == 0, 1],
    color="red",
    marker="x",
    s=100,
    label="Rejected"
)

plt.scatter(
    X_scaled[y == 1, 0],
    X_scaled[y == 1, 1],
    color="green",
    marker="o",
    s=100,
    label="Approved"
)
```

```
x_min = X_scaled[:, 0].min() - 0.5
x_max = X_scaled[:, 0].max() + 0.5

x_values = np.linspace(x_min, x_max, 100)

if model.weights[1] != 0:
    y_values = (
        -(model.weights[0] * x_values + model.bias)
        / model.weights[1]
    )

    plt.plot(
        x_values,
        y_values,
        color="black",
        linewidth=2,
        label="Decision Boundary"
    )

plt.xlabel("Annual Income (Scaled)")
plt.ylabel("Credit Score (Scaled)")
plt.title("Loan Approval - Perceptron Decision Boundary")
plt.legend()
plt.grid(True)
plt.show()

learning_rates = [0.001, 0.01, 0.1, 1.0]

learning_rate_results = []

for lr in learning_rates:

    temp_model = Perceptron(
        learning_rate=lr,
        epochs=100
    )
```

```
    temp_model.fit(X_scaled, y)

    temp_predictions = temp_model.predict(X_scaled)

    temp_accuracy = np.mean(
        temp_predictions == y
    ) * 100

    learning_rate_results.append({
        "Learning Rate": lr,
        "Epochs Used": len(temp_model.errors),
        "Accuracy (%)": temp_accuracy
    })

lr_results_df = pd.DataFrame(learning_rate_results)

print("\n" + "=" * 60)
print("EFFECT OF LEARNING RATE")
print("=" * 60)

print(lr_results_df.to_string(index=False))

epoch_values = [1, 5, 10, 20, 50, 100]

epoch_results = []

for ep in epoch_values:

    temp_model = Perceptron(
        learning_rate=0.01,
        epochs=ep
    )

    temp_model.fit(X_scaled, y)

    temp_predictions = temp_model.predict(X_scaled)

    temp_accuracy = np.mean(
```

```
            temp_predictions == y
        ) * 100

        epoch_results.append({
            "Epochs": ep,
            "Accuracy (%)": temp_accuracy
        })

epoch_results_df = pd.DataFrame(epoch_results)

print("\n" + "=" * 60)
print("EFFECT OF NUMBER OF EPOCHS")
print("=" * 60)

print(epoch_results_df.to_string(index=False))

plt.figure(figsize=(8, 5))

plt.plot(
    lr_results_df["Learning  Rate"],
    lr_results_df["Accuracy  (%)"],
    marker="o",
    linewidth=2
)

plt.xscale("log")
plt.xlabel("Learning Rate")
plt.ylabel("Accuracy (%)")
plt.title("Effect of Learning Rate on Accuracy")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))

plt.plot(
    epoch_results_df["Epochs"],
    epoch_results_df["Accuracy (%)"],
    marker="o",
```

```
    color="purple",
    linewidth=2
)

plt.xlabel("Number of Epochs")
plt.ylabel("Accuracy (%)")
plt.title("Effect of Epochs on Accuracy")
plt.grid(True)
plt.show()

new_income = 72000
new_credit_score = 725

new_application = np.array([
    [new_income, new_credit_score]
])

new_application_scaled = (
    new_application - X_mean
) / X_std

new_prediction = model.predict(
    new_application_scaled
)[0]

print("\n" + "=" * 60)
print("NEW LOAN APPLICATION")
print("=" * 60)

print("Annual Income :", new_income)
print("Credit Score  :", new_credit_score)

if new_prediction == 1:
    print("Prediction      : LOAN APPROVED")
else:
    print("Prediction      : LOAN REJECTED")
```

OUTPUT :-

```
...  ============================================================
     LOAN APPROVAL DATASET
     ============================================================
      Annual_Income  Credit_Score  Loan_Status
              25000           550            0
              30000           580            0
              35000           600            0
              40000           610            0
              42000           620            0
              45000           630            0
              48000           640            0
              50000           650            1
              52000           660            1
              55000           670            1
              58000           680            1
              60000           690            1
              62000           700            1
              65000           710            1
              68000           720            1
              70000           730            1
              75000           740            1
              80000           750            1
              85000           760            1
              90000           780            1

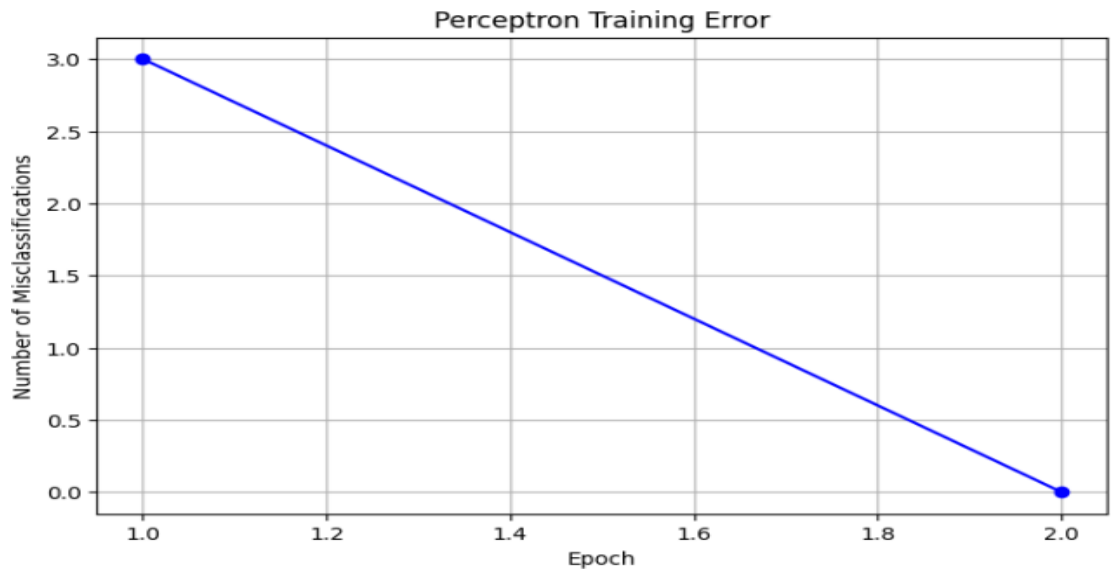
     ============================================================
     MODEL RESULTS
     ============================================================
     Learning Rate : 0.01
     Maximum Epochs: 100
     Actual Epochs : 2

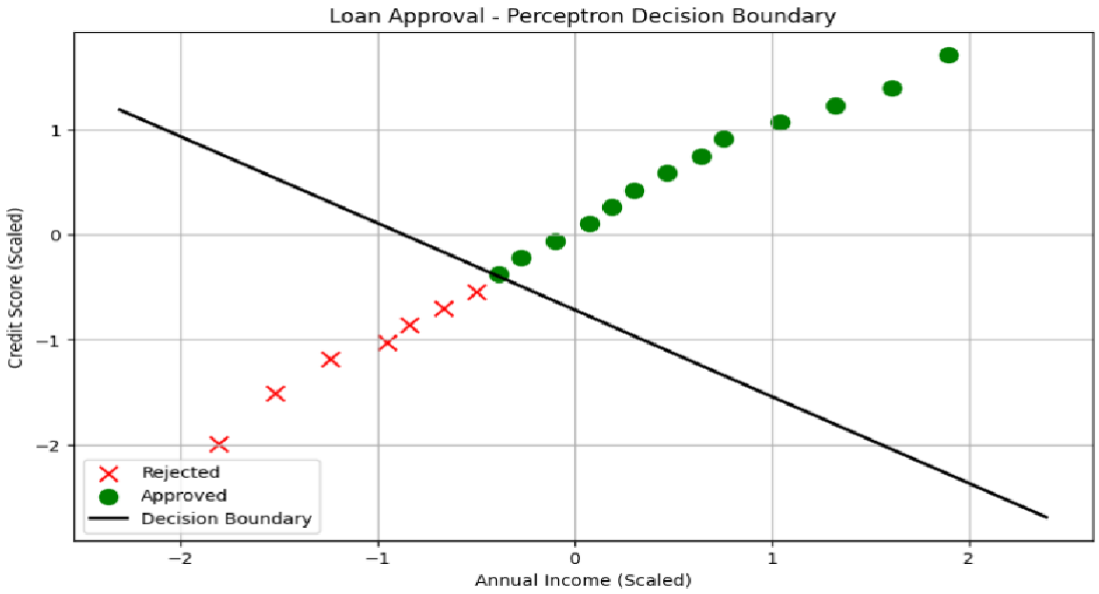
     Final Weights:
     [0.01153126 0.01395388]

     Bias:
     0.01

     Accuracy: 100.00%
```

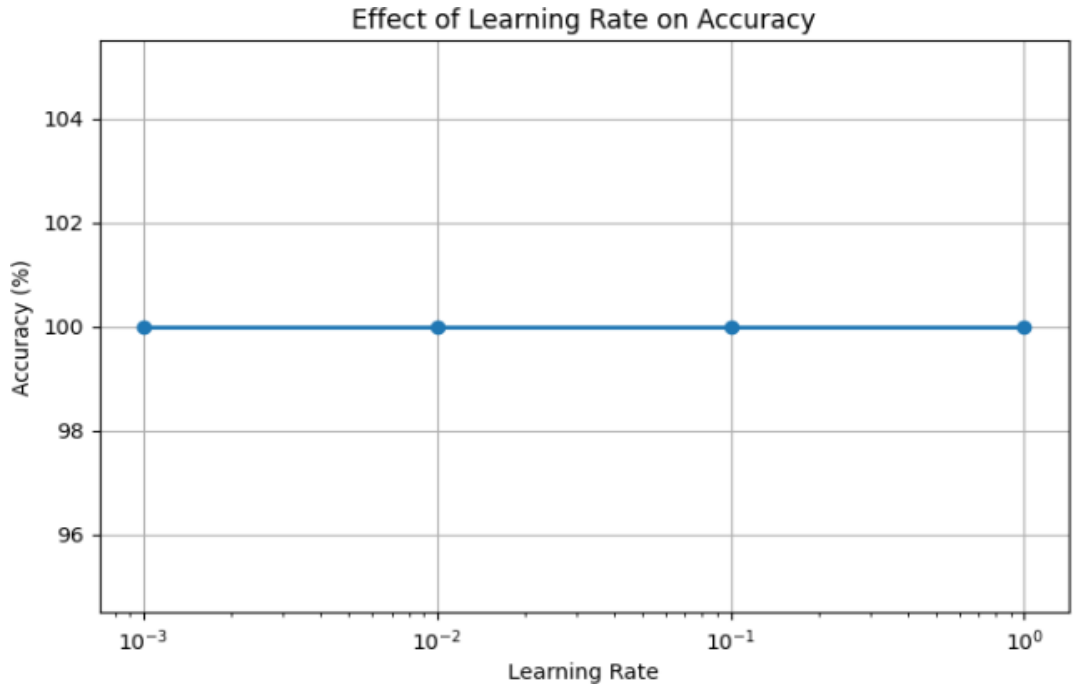
```
============================================================
PREDICTIONS
============================================================
 Annual_Income  Credit_Score  Loan_Status  Predicted_Status Prediction_Result
         25000           550            0                 0           Correct
         30000           580            0                 0           Correct
         35000           600            0                 0           Correct
         40000           610            0                 0           Correct
         42000           620            0                 0           Correct
         45000           630            0                 0           Correct
         48000           640            0                 0           Correct
         50000           650            1                 1           Correct
         52000           660            1                 1           Correct
         55000           670            1                 1           Correct
         58000           680            1                 1           Correct
         60000           690            1                 1           Correct
         62000           700            1                 1           Correct
         65000           710            1                 1           Correct
         68000           720            1                 1           Correct
         70000           730            1                 1           Correct
         75000           740            1                 1           Correct
         80000           750            1                 1           Correct
         85000           760            1                 1           Correct
         90000           780            1                 1           Correct
```

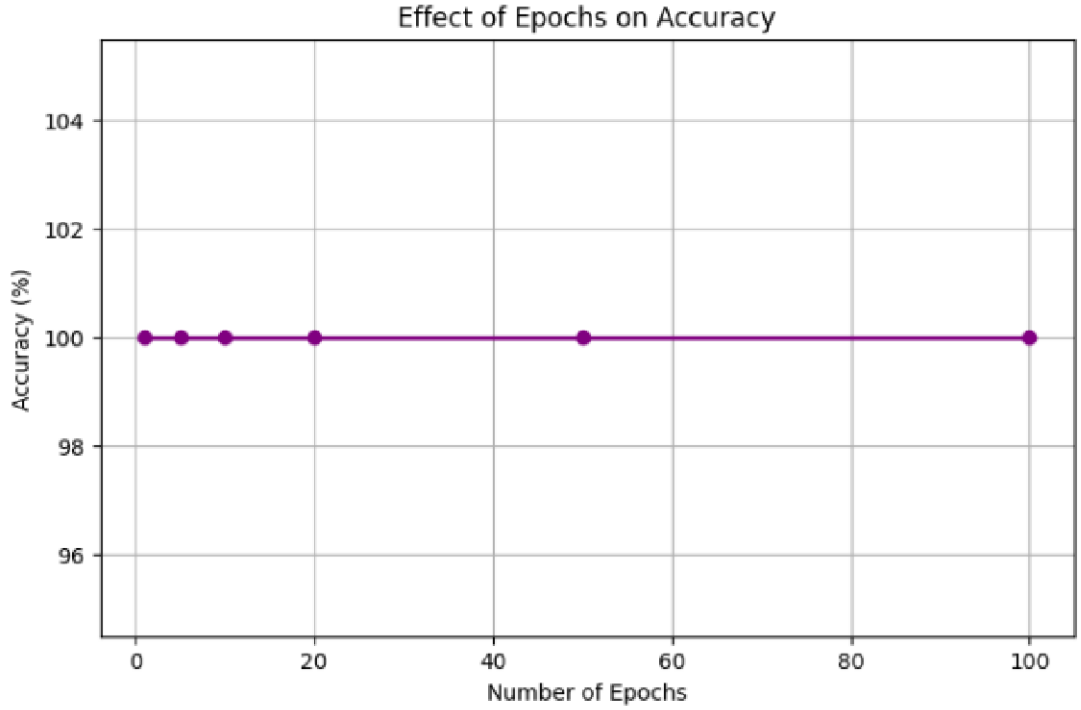





```
============================================================
EFFECT OF LEARNING RATE
============================================================
 Learning Rate  Epochs Used  Accuracy (%)
         0.001            2         100.0
         0.010            2         100.0
         0.100            2         100.0
         1.000            2         100.0

============================================================
EFFECT OF NUMBER OF EPOCHS
============================================================
 Epochs  Accuracy (%)
      1         100.0
      5         100.0
     10         100.0
     20         100.0
     50         100.0
    100         100.0
```






```
============================================================
NEW LOAN APPLICATION
============================================================
Annual Income : 72000
Credit Score  : 725
Prediction    : LOAN APPROVED
```

PRACTICAL :-2

Practical 2.1:-

Adaptive Linear Neuron (ADALINE) for House Price Prediction

Problem Statement:-

Implement an ADALINE network using the Delta Learning Rule to predict house prices based on area, number of rooms, and location score.

Objectives:-

Understand Widrow-Hoff learning.

Compare ADALINE with Perceptron.

Study Mean Squared Error minimization.

Tools:-

Python, Pandas, NumPy

Source Code:-

```
import numpy as np
import pandas as pd

# Sample dataset
data = {
    "Area": [1000, 1200, 1500, 1800, 2000],
    "Rooms": [2, 3, 3, 4, 4],
    "Location": [5, 6, 7, 8, 9],
    "Price": [200, 250, 300, 360, 400]
}

df = pd.DataFrame(data)

# Input and Output
X = df[["Area", "Rooms", "Location"]].values
y = df["Price"].values
```

```python
# Normalize data
X = X / X.max(axis=0)
y = y / y.max()

# Add bias
X = np.c_[np.ones(len(X)), X]

# ADALINE Training
weights = np.zeros(X.shape[1])
lr = 0.1
epochs = 50

for epoch in range(epochs):
    output = np.dot(X, weights)
    error = y - output
    weights += lr * np.dot(X.T, error) / len(X)

# Predictions
pred = np.dot(X, weights)

print("Actual Price\tPredicted Price")
for a, p in zip(y, pred):
    print(f"{a:.3f}\t\t{p:.3f}")

mse = np.mean((y - pred) ** 2)
print("\nFinal Weights:", weights)
print("Mean Squared Error:", round(mse, 5))
```

OUTPUT :-

```
Actual Price    Predicted Price
0.500           0.583
0.625           0.691
0.750           0.752
0.900           0.871
1.000           0.920

Final Weights: [0.22079069 0.23233283 0.23838089 0.22851687]
Mean Squared Error: 0.0037
```

2.2 Loan Approval Prediction:-

Problem Statement:-

A financial institution wants to automate the loan approval process based on customer information such as annual income, credit score, employment experience, and existing loan amount. Develop an ADALINE network using the Delta Learning Rule to classify loan applications as **Approved (1)** or **Rejected (0).**

Features:-

- **Annual Income**
- **Credit Score**
- **Years of Employment**
- **Existing Loan Amount**

Output:-

- **1 → Approved**
- **0 → Rejected**

Source Code:-

```
import numpy as np
import pandas as pd
# Loan Approval Dataset
data = {
    "Income": [30,40,50,60,70,80,90],
    "CreditScore": [550,600,620,680,700,780,820],
    "Experience": [1,2,3,4,5,6,7],
    "LoanAmount": [25,30,40,45,50,55,60],
    "Approval": [0,0,0,1,1,1,1]
}
df = pd.DataFrame(data)
print(df)
# Input and Output
X = df[["Income", "CreditScore", "Experience", "LoanAmount"]].values
y = df["Approval"].values
```

```
# Normalize Inputs
import numpy as np

# Normalize Inputs
X = X / X.max(axis=0)

# Add Bias Column
X = np.c_[np.ones(len(X)), X]

# ADALINE Parameters
weights = np.zeros(X.shape[1])
learning_rate = 0.1
epochs = 30

print("\nInitial Weights")
print(weights)

# Training
for epoch in range(epochs):
    output = np.dot(X, weights)
    error = y - output
    weights += learning_rate * np.dot(X.T, error) / len(X)
    mse = np.mean(error ** 2)

    if (epoch + 1) % 5 == 0:
        print(f"Epoch {epoch+1:2d}  MSE = {mse:.4f}")

# Prediction
prediction = np.dot(X, weights)

# Threshold
predicted_class = (prediction >= 0.5).astype(int)

print("\nFinal Weights")
print(weights)

print("\nPrediction Results")
print("Actual\tOutput\tPredicted Class")
```

```
for actual, out, pred in zip(y, prediction, predicted_class):
    print(f"{actual}\t{out:.3f}\t{pred}")

accuracy = np.mean(predicted_class == y) * 100

print("\nFinal Mean Squared Error :", round(np.mean((y - prediction) ** 2), 4))
print("Accuracy :", round(accuracy, 2), "%")
```

OUTPUT :-

```
...      Income  CreditScore  Experience  LoanAmount  Approval
     0      30          550           1          25         0
     1      40          600           2          30         0
     2      50          620           3          40         0
     3      60          680           4          45         1
     4      70          700           5          50         1
     5      80          780           6          55         1
     6      90          820           7          60         1

     Initial Weights
     [0. 0. 0. 0. 0.]
     Epoch  5  MSE = 0.1762
     Epoch 10  MSE = 0.1480
     Epoch 15  MSE = 0.1409
     Epoch 20  MSE = 0.1347
     Epoch 25  MSE = 0.1290
     Epoch 30  MSE = 0.1238

     Final Weights
     [0.04797937 0.22528617 0.13332079 0.27594526 0.20808714]

     Prediction Results
     Actual  Output  Predicted Class
     0       0.339   0
     0       0.429   0
     0       0.531   1
     1       0.622   1
     1       0.708   1
     1       0.802   1
     1       0.891   1

     Final Mean Squared Error : 0.1228
     Accuracy : 85.71 %
```

PRACTICAL :-3

Practical 3:-

Problem Statement:-

A smart traffic management system receives two fuzzy inputs:

- **Degree of Traffic Congestion**
- **Degree of Road Occupancy**

Both values are represented as membership values between 0 and 1.
Implement fuzzy logic operators to assist the traffic controller in evaluating road conditions.

Input:-

Traffic Congestion = 0.75
Road Occupancy = 0.62

Output:-

Traffic AND Occupancy
Traffic OR Occupancy
NOT Traffic
NOT Occupancy

Source Code:-

```
import matplotlib.pyplot as plt

print("=" * 60)
print("      SMART TRAFFIC MANAGEMENT USING FUZZY LOGIC")
print("=" * 60)
print("\nEnter membership values between 0 and 1.")

# Input
traffic = float(input("Traffic Congestion Membership (0-1): "))
occupancy = float(input("Road Occupancy Membership (0-1): "))

# Validation
if not (0 <= traffic <= 1 and 0 <= occupancy <= 1):
    print("\nError! Membership values must be between 0 and 1.")
    exit()
```

```
# ------------------------
# Fuzzy Operations
# ------------------------
fuzzy_and = min(traffic, occupancy)
fuzzy_or = max(traffic, occupancy)
not_traffic = 1 - traffic
not_occupancy = 1 - occupancy

# ------------------------
# Display Results
# ------------------------
print("\n" + "=" * 60)
print("              FUZZY OPERATION RESULTS")
print("=" * 60)
print(f"Traffic Congestion Membership : {traffic:.2f}")
print(f"Road Occupancy Membership     : {occupancy:.2f}")
print("\nFuzzy AND (Minimum) :", round(fuzzy_and, 2))
print("Fuzzy OR (Maximum)  :", round(fuzzy_or, 2))
print("NOT Traffic                 :", round(not_traffic, 2))
print("NOT Occupancy               :", round(not_occupancy, 2))

print("\nInterpretation")
print("-" * 60)
if fuzzy_and > 0.7:
    print("Road is highly congested and highly occupied.")
elif fuzzy_and > 0.4:
    print("Road has moderate congestion.")
else:
    print("Road congestion is comparatively low.")

# ------------------------
# Visualization
# ------------------------
labels = [
    "Traffic",
    "Occupancy",
    "AND",
```

```
    "OR",
    "NOT Traffic",
    "NOT Occupancy",
]
values = [
    traffic,
    occupancy,
    fuzzy_and,
    fuzzy_or,
    not_traffic,
    not_occupancy,
]

plt.figure(figsize=(8, 5))
bars = plt.bar(labels, values)
plt.ylim(0, 1.1)
plt.ylabel("Membership Value")
plt.xlabel("Fuzzy Operations")
plt.title("Smart Traffic Control using Fuzzy Logic")

# Show values on top of bars
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height + 0.02,
        f"{height:.2f}",
        ha="center",
    )

plt.grid(axis="y", linestyle="--")
plt.show()
```

OUTPUT:-

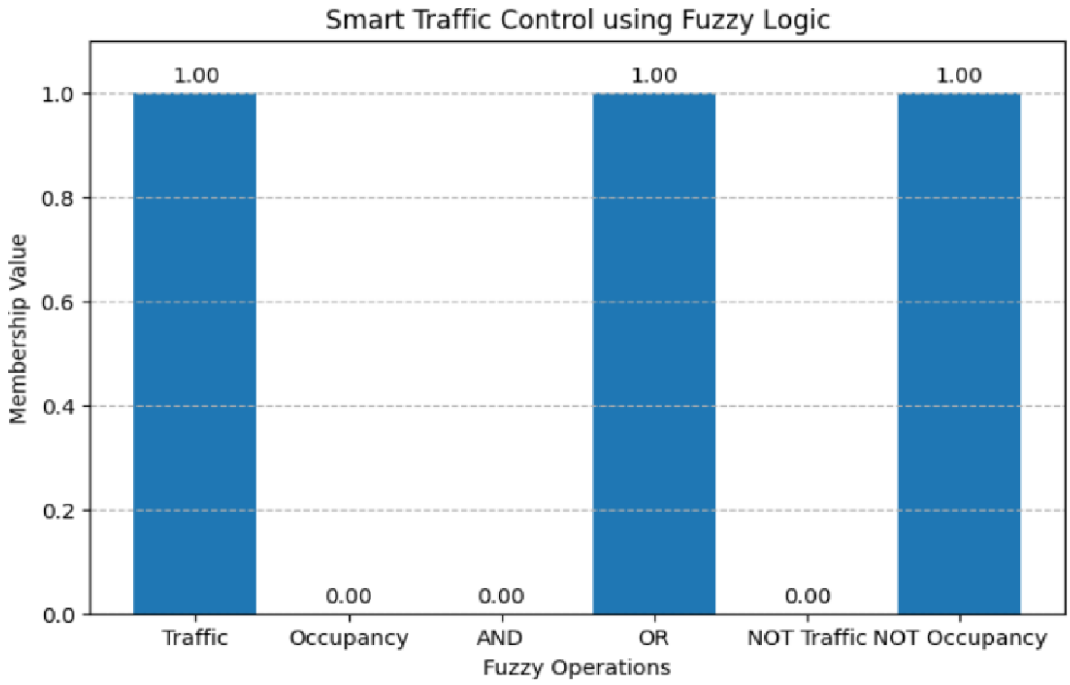
```
============================================================
      SMART TRAFFIC MANAGEMENT USING FUZZY LOGIC
============================================================

Enter membership values between 0 and 1.
Traffic Congestion Membership (0-1): 1
Road Occupancy Membership (0-1): 0

============================================================
                FUZZY OPERATION RESULTS
============================================================
Traffic Congestion Membership : 1.00
Road Occupancy Membership     : 0.00

Fuzzy AND (Minimum) : 0.0
Fuzzy OR (Maximum)  : 1.0
NOT Traffic                   : 0.0
NOT Occupancy                 : 1.0

Interpretation
------------------------------------------------------------
Road congestion is comparatively low.
```



Practical-4

Pratical Source Code:-

Q1] Classical Sets and Fuzzy Sets

Let the universal set be $X = \{1,2,3,4,5,6,7,8,9,10\}$

A fuzzy Set A is given by

$A = \{0.1/1, 0.3/2, 0.5/3, 0.7/4, 0.9/5, 1/6, 0.8/7, 0.6/8, 0.4/9, 0.2/10\}$

Find :-

(i) The Support of A

(ii) The Core of A

(iii) The height of A

(iv) The α-cut of A for $\alpha = 0.5$.

Soln :- Given :- Universal Set $X = \{1,2,3,4,5,6,7,8,9,10\}$

Fuzzy Set $A = \{0.1/1, 0.3/2, 0.5/3, 0.7/4, 0.9/5, 1/6, 0.8/7, 0.6/8, 0.4/9, 0.2/10\}$

1] Support of A

Formula : $\text{Support}(A) = \{x \in X \mid \mu_A(x) > 0\}$

From A, all elements have membership > 0.

$\therefore \text{Support}(A) = \{1,2,3,4,5,6,7,8,9,10\}$

2] Core of A

Formula : $\text{Core}(A) = \{x \in X \mid \mu_A(x) = 1\}$

From A, $\mu_A(6) = 1$ and no other element has membership 1.

$\therefore \text{Core}(A) = \{6\}$

3] Height of A

formula : $\text{Height}(A) = \sup\{\mu_A(x) \mid x \in X\}$

= maximum membership value in A

From A, the maximum membership value is 1 (at $x = 6$)

$\therefore \text{Height}(A) = 1$

4] α-cut of A for $\alpha = 0.5$

Formula :- $A_\alpha = \{x \in X \mid \mu_A(x) \geq \alpha\}$

Here $\alpha = 0.5 \Rightarrow A_{0.5} = \{x \in X \mid \mu_A(x) \geq 0.5\}$

From A, the elements with membership ≥ 0.5 are:

3(0.5), 4(0.7), 5(0.9), 6(1), 7(0.8), 8(0.6)

$\therefore A_{0.5} = \{3, 4, 5, 6, 7, 8\}$

Summary of Answers:

1] Support(A) = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10} 2] Core(A) = {6}

3] Height(A) = 1 4] $A_{0.5} = \{3, 4, 5, 6, 7, 8\}$

Q2] Cartesian Product and Fuzzy Relations

Let $A = \{0.2/x_1, 0.6/x_2, 0.9/x_3\}$

and

$B = \{0.4/y_1, 0.7/y_2\}$

Find the Fuzzy Cartesian product A×B, where

$\mu_{A \times B}(x, y) = \min[\mu_A(x), \mu_B(y)]$.

Represent your answer as a membership matrix.

Soln :- $A = \{0.2/x_1, 0.6/x_2, 0.9/x_3\}$

$B = \{0.4/y_1, 0.7/y_2\}$

To find: Fuzzy Cartesian Product A×B

Formula : $\mu_{A \times B}(x_i, y_j) = \min[\mu_A(x_i), \mu_B(y_j)]$

(Take minimum of membership values)

Step-1 : List the membership values.

For set A	For Set B
$\mu_A(x_1) = 0.2$	$\mu_B(y_1) = 0.4$

$\mu_A(x_2) = 0.6$ $\mu_B(y_2) = 0.7$
$\mu_A(x_3) = 0.9$

Step-2: Use the formula $\mu_{A\times B}(x_i, y_j) = \min[\mu_A(x_i), \mu_B(y_j)]$
Compute for each pair (x_i, y_j).

(x_1, y_1) : $\min(0.2, 0.4) = 0.2$
(x_1, y_2) : $\min(0.2, 0.7) = 0.2$
(x_2, y_1) : $\min(0.6, 0.4) = 0.4$
(x_2, y_2) : $\min(0.6, 0.7) = 0.6$
(x_3, y_1) : $\min(0.9, 0.4) = 0.4$
(x_3, y_2) : $\min(0.9, 0.7) = 0.7$

Step-3: Write the results in a membership matrix.
Rows → elements of A (x_1, x_2, x_3)
Columns → elements of B (y_1, y_2)

Membership Matrix of A×B

A×B	$y_1(0.4)$	$y_2(0.7)$
$x_1(0.2)$	0.2	0.2
$x_2(0.6)$	0.4	0.6
$x_3(0.9)$	0.4	0.7

Step-4: List the fuzzy Cartesian product as a set of ordered pairs (optional)

$A\times B = \{0.2/(x_1, y_1), 0.2/(x_2, y_2), 0.4/(x_2, y_1), 0.6/(x_2, y_2), 0.4/(x_3, y_1), 0.7/(x_3, y_2)\}$

Final Answer :- The membership matrix of AXB is

AXB	$y_1(0.4)$	$y_2(0.7)$
$x_1(0.2)$	0.2	0.2
$x_2(0.6)$	0.4	0.6
$x_3(0.9)$	0.4	0.7

Q3) Composition of fuzzy Relations

Consider two fuzzy relations:

$$R = \begin{bmatrix} 0.2 & 0.5 & 0.8 \\ 0.7 & 0.4 & 0.3 \end{bmatrix}$$

and

$$S = \begin{bmatrix} 0.6 & 0.9 \\ 0.5 & 0.2 \\ 0.8 & 0.4 \end{bmatrix}$$

Calculate the max-min composition.

$$T = R \circ S.$$

Soln :- $T = \begin{matrix} T_{11} & T_{12} \\ T_{21} & T_{22} \end{matrix}$

1) Find T_{11} (Row 1 of R with Column 1 of S)

- Row 1 of R : $[0.2 \quad 0.5 \quad 0.8]$
- Column 1 of S : $\begin{bmatrix} 0.6 \\ 0.5 \\ 0.8 \end{bmatrix}$

1. Pairwise minimums :

$\min(0.2, 0.6) = 0.2$

$\min(0.5, 0.5) = 0.5$

$\min(0.8, 0.8) = 0.8$

2. Take maximum of these minimums

$T_{11} = \max(0.2, 0.5, 0.8) = 0.8$

2] Find T_{12} (Row 1 of R with column 2 of S)

- Row 1 of R: $[0.2 \quad 0.5 \quad 0.8]$
- Column 2 of S: $\begin{bmatrix} 0.9 \\ 0.2 \\ 0.4 \end{bmatrix}$

1. Pairwise minimums:

$\min(0.2, 0.9) = 0.2$

$\min(0.5, 0.2) = 0.2$

$\min(0.8, 0.4) = 0.4$

2. Take maximum of these minimums:

$T_{12} = \max(0.2, 0.2, 0.4) = 0.4$

3] Finding T_{21} (Row 2 of R with Column 1 of S)

- Row 2 of R: $[0.7 \quad 0.4 \quad 0.3]$
- Column 1 of S: $\begin{bmatrix} 0.6 \\ 0.5 \\ 0.8 \end{bmatrix}$

1. Pairwise minimums:

$\min(0.7, 0.6) = 0.6$

$\min(0.4, 0.5) = 0.4$

$\min(0.3, 0.8) = 0.3$

2. Take minimum of these minimums:

$T_{21} = \max(0.6, 0.4, 0.3) = 0.6$

4) Finding T_{22} (Row 2 of R with Column 2 of S)

- Row 2 of R : $[0.7 \quad 0.4 \quad 0.3]$
- Column 2 of S : $\begin{bmatrix} 0.9 \\ 0.2 \\ 0.4 \end{bmatrix}$

1. Pairwise minimums:

$\min(0.7, 0.9) = 0.7$

$\min(0.4, 0.2) = 0.2$

$\min(0.3, 0.4) = 0.3$

2. Take maximum of these minimums:

$T_{22} = \max(0.7, 0.2, 0.3) = 0.7$

Final Answer:

$$T = R \circ S = \begin{bmatrix} 0.8 & 0.4 \\ 0.6 & 0.7 \end{bmatrix}$$

PRACTICAL :- 5

Practical-5

AIM:-To develop a customer segmentation model using a Self-Organizing Map (SOM) by transforming raw retail transaction data into customer behavioral features, training an unsupervised neural network to discover natural customer groups, and interpreting the identified segments for business decision-making.

Source Code:-

```
import pandas as pd import
numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler from
sklearn.cluster import KMeans
from minisom import MiniSom


excel_file = pd.ExcelFile('online_retail_II.xlsx') print(excel_file.sheet_names)

df1 = pd.read_excel(excel_file, sheet_name='Year 2009-2010') df2 =
pd.read_excel(excel_file, sheet_name='Year 2010-2011') df = pd.concat([df1, df2],
ignore_index=True)

df.head() df.shape
df.info() df.describe()
df.columns
df.isnull().sum()

df = df.dropna(subset=['Customer ID'])

df = df[~df['Invoice'].astype(str).str.startswith('C')] df = df[df['Quantity'] > 0]

df = df[df['Price'] > 0]
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate']) df['TotalAmount'] =
df['Quantity'] * df['Price']
snapshot_date = df['InvoiceDate'].max() + pd.Timedelta(days=1) recency = (snapshot_date -
df.groupby('Customer ID')['InvoiceDate'].max()).dt.days
frequency = df.groupby('Customer ID')['Invoice'].nunique()
```

```
monetary = df.groupby('Customer ID')['TotalAmount'].sum() avg_order_value = monetary /
frequency
unique_products = df.groupby('Customer ID')['StockCode'].nunique() total_quantity =
df.groupby('Customer ID')['Quantity'].sum() customer_data = pd.DataFrame({
'Recency': recency, 'Frequency':
frequency, 'Monetary': monetary,
'AvgOrderValue': avg_order_value, 'UniqueProducts':
unique_products, 'TotalQuantity': total_quantity
})

customer_data.reset_index(inplace=True)

customer_data.head()
customer_data.describe()

X = customer_data.drop(columns=['Customer ID']) scaler =

MinMaxScaler()

X_scaled = scaler.fit_transform(X)
!pip install minisom som =

MiniSom(

x=10,

y=10,

input_len=X_scaled.shape[1],

sigma=1.0, learning_rate=0.5, random_seed=42)
som.random_weights_init(X_scaled) som.train_random(

data=X_scaled,

num_iteration=5000) plt.figure(figsize=(10,8))
plt.pcolor(som.distance_map().T, cmap='bone_r') plt.colorbar()
plt.title("U-Matrix") plt.show()
winner_coordinates = np.array( [som.winner(x) for x in
X_scaled])
customer_data["SOM_X"] = winner_coordinates[:,0]
```

```
customer_data["SOM_Y"] = winner_coordinates[:,1]

from collections import Counter
hits = Counter(map(tuple, winner_coordinates))

print(hits)
weights = som.get_weights()
weight_vectors = weights.reshape(-1, weights.shape[2]) kmeans =
KMeans(n_clusters=4, random_state=42)

kmeans.fit(weight_vectors) neuron_clusters =

kmeans.labels_

neuron_clusters = neuron_clusters.reshape(10,10) customer_clusters =

[]

for customer in X_scaled:
    x, y = som.winner(customer) customer_clusters.append(neuron_clusters[x, y])

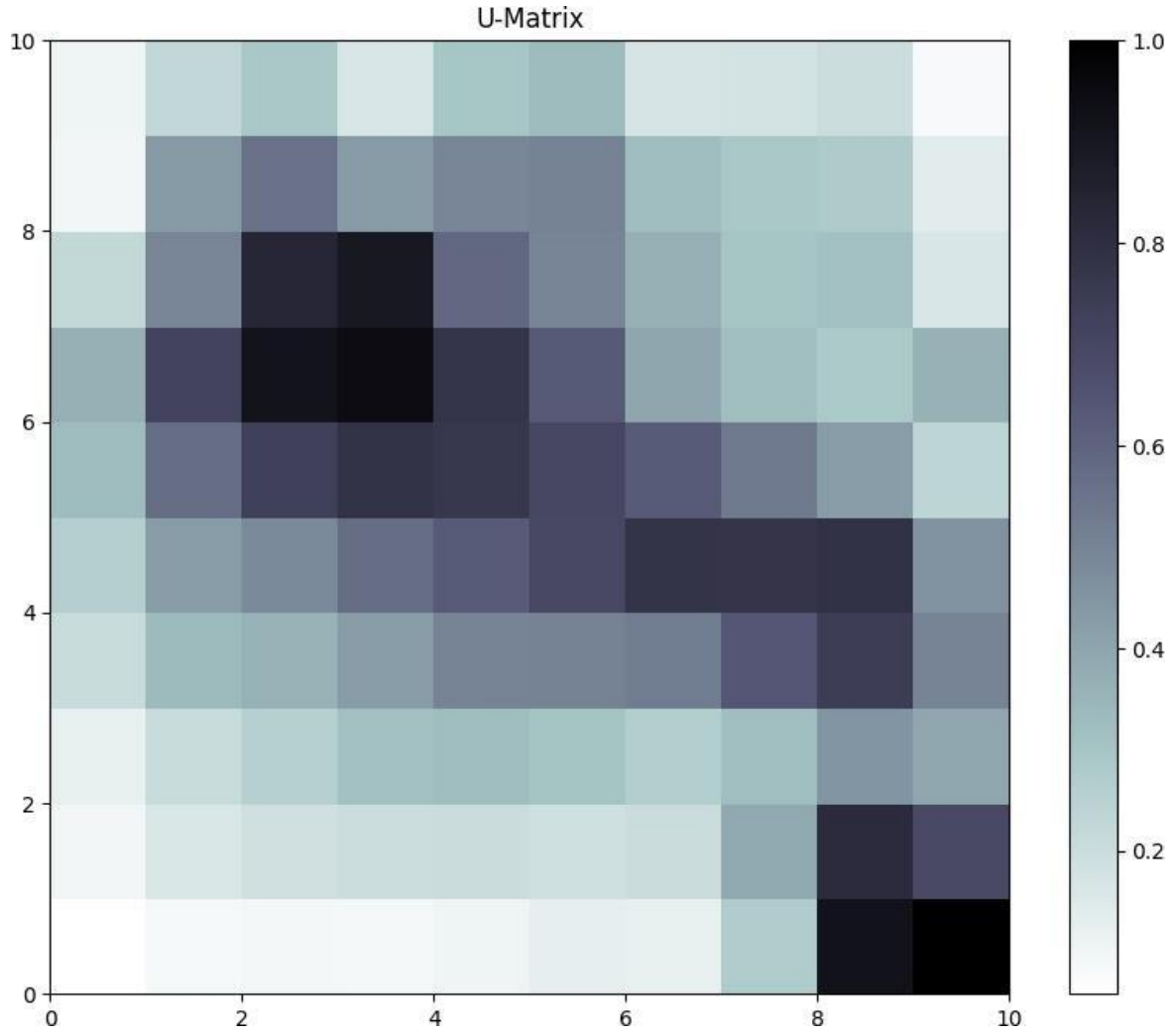

customer_data["Cluster"] = customer_clusters customer_data.groupby("Cluster").mean(numeric_only=True)
customer_data["Cluster"].value_counts() customer_data.to_csv("Customer_Segments.csv",index=False)

customer_data.head()
```

OUTPUT:-

```
['Year 2009-2010', 'Year 2010-2011']
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1067371 entries, 0 to 1067370
Data columns (total 8 columns):
 #   Column       Non-Null Count    Dtype
---  -------      --------------    -----
 0   Invoice      1067371 non-null  object
 1   StockCode    1067371 non-null  object
 2   Description  1062989 non-null  object
 3   Quantity     1067371 non-null  int64
 4   InvoiceDate  1067371 non-null  datetime64[ns]
```

```
 5   Price        1067371 non-null  float64
 6   Customer ID  824364 non-null   float64
 7   Country      1067371 non-null  object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 65.1+ MB
```



	Customer ID	Recency	Frequency	Monetary	AvgOrder Value	Unique Products	Total Quantity	SOM_X	SOM_Y	Cluster
0	12346.0	326	12	77556.46	6463.038	27	74285	9	6	2
1	12347.0	2	8	5633.32	704.1650	126	3286	4	0	3
2	12348.0	75	5	2019.40	403.8800	25	2714	2	2	3
3	12349.0	19	4	4428.69	1107.1725	138	1624	5	1	3
4	12350.0	310	1	334.40	334.4000	17	197	9	7	1

PRACTICAL NO:06

PRACTICAL 6:-

AIM:-

Experimental Analysis of a Hopfield Neural Network for Binary Pattern Storage, Associative Retrieval, Energy Minimization, and Noise Tolerance.

SourceCode:

```
import numpy as np

import matplotlib.pyplot as plt


patterns = {

  "A": np.array([

    [-1, 1, 1, 1, -1],

    [ 1, -1, -1, -1, 1],

    [ 1, 1, 1, 1, 1],

    [ 1, -1, -1, -1, 1],

    [ 1, -1, -1, -1, 1]

  ]),

  "B": np.array([

    [ 1, 1, 1, 1, -1],

    [ 1, -1, -1, -1, 1],

    [ 1, 1, 1, 1, -1],

    [ 1, -1, -1, -1, 1],

    [ 1, 1, 1, 1, -1]

  ]),

  "C": np.array([

    [-1, 1, 1, 1, 1],

    [ 1, -1, -1, -1, -1],

    [ 1, -1, -1, -1, -1],
```

```
    [ 1, -1, -1, -1, -1],
    [-1, 1, 1, 1, 1]
  ])

}

  def
    display_pattern(pattern,
    title):
    plt.figure(figsize=(3,
    3)) plt.imshow(
    pattern,
    cmap="gray
    ", vmin=-
    1, vmax=1
  )
  plt.title(titl
  e)
  plt.axis("off"
  ) plt.show()

print("=" * 60)
print(" HOPFIELD NEURAL NETWORK")
print(" INTERACTIVE LEARNING
SYSTEM")
print("=" * 60)

print("\nThe network has learned these
patterns:") for name, pattern in
patterns.items():
```

```
  print("\nPattern",
  name)
  print(pattern)

pattern_vectors = []

  for pattern in
    patterns.values():
    pattern_vectors.append(
    pattern.flatten()
  )

number_of_neurons = pattern_vectors[0].size W
= np.zeros(
  (number_of_neurons, number_of_neurons)
)

  for pattern in pattern_vectors: W
    +=
    np.outer(
    pattern,
    pattern
  )

np.fill_diagonal(W, 0)

W = W / number_of_neurons

print("\n                          ")
print("NETWORK INFORMATION")
```

```
print("                              ")
print("Number of stored patterns :",
len(patterns)) print("Image size : 5 ×
5")
print("Number of neurons :",
number_of_neurons) print("Weight matrix
:", W.shape)
  def
    calculate_energy(st
    ate): state =
    state.flatten()
    energy = -0.5 *
    np.dot(
      state,
    np.dot(W,
    state)

  )
  return energy

  def
    update_network(stat
    e):   new_state   =
    state.copy()  for  i
    in
    range(len(state)):
      activation =
      np.dot( W[i],
      new_state
    )
      if activation >=
```

```
    0: new_state[i] =
    1
  else:
    new_state[i] = -1
return new_state


def retrieve_pattern(input_pattern, max_iterations=20): state
  =
  input_pattern.flatten()
  .copy() energy_history
  = [
    calculate_energy(state)
]
  for        iteration        in
  range(max_iterations):
  new_state                    =
  update_network(state)
  energy_history.append(
    calculate_energy(new_state)
  )


    if
    np.array_equa
    l( new_state,


    state
  ):
    return (
      new_state.reshape(5
```

```
      , 5),
      energy_history,
      iteration + 1
    )
  state = new_state return
(
  state.reshape(5
  , 5),
  energy_history,
  max_iterations
)


def calculate_accuracy(original,
  retrieved): correct = np.sum(
  original == retrieved
)
total = original.size
  return ( correct /
  total
) * 100
def add_noise(pattern, noise_percentage): noisy = pattern.copy()
total_pixels = noisy.size
number_of_pixels = int(
  total_pixels *
  noise_percentage /
  100

)
  if number_of_pixels
  == 0: return noisy
  indices =
```

```
      np.random.choice(
      total_pixels,
      number_of_pixels,
      replace=False
    )
    flat =
    noisy.flatten()
    flat[indices] *= -1
    return flat.reshape(5, 5)


    def
      display_result
      (
      input_pattern,
      retrieved_patt
      ern, title
  ):
    plt.figure(figsize=(9, 3))
    plt.subplot(1,
    2, 1)
    plt.imshow(
      input_pattern
      ,
      cmap="gray",
      vmin=-1,
      vmax=1
    )
    plt.title("Input
    Pattern")
    plt.axis("off")
```

```
plt.subplot(1, 2,
2) plt.imshow(


  retrieved_pat
  tern,
  cmap="gray",
  vmin=-1,
  vmax=1
)
plt.title("Hopfield
               Retrieved
               Pattern")
plt.axis("off")
plt.suptitle(t
itle)
plt.tight_layo
ut()
plt.show()


while True:
  print("\n")
print("=" * 60)
print(" MAIN MENU")
print("=" * 60)
print("1. Test a stored pattern
with noise") print("2. Enter
your own 5×5 pattern")
print("3. Display stored
patterns") print("4. Exit")
```

```
choice = input(
  "\nEnter your choice (1-4): "
)
if choice == "1":
  print("\nAvailable stored
  patterns:") for name in
  patterns:


    print(" ", name)


  pattern_name = input(
    "\nWhich pattern do you want to test? "
  ).upper()
    if pattern_name not in
    patterns: print(
      "\nInvalid pattern name."
    )
    continue


try:
     noise =
       float(
       input(
       "Enter noise
       percentage " "(0-
       80): "
```

```
    )
  )
 except:
  print(
    "\nPlease enter a valid number."
  )
  continue


  if noise < 0 or noise > 80:
  print(
    "\nNoise must be between 0 and 80."
  )
  continue

  original =
  patterns[
  pattern_name
]
  noisy_input =
    add_noise(
    original,
  noise
)


  retrieved, energy_history,
    iterations = ( retrieve_pattern(
    noisy_input
  )
)
```

```
  accuracy =
  calculate_accuracy(
  original, retrieved
)


print("\n")
print("=" * 60)
print(" RETRIEVAL RESULT")
print("=" *
60) print(
  "\nOriginal Pattern:"
)
print(origina
l) print(


  "\nNoisy Input:"
)
print(noisy_input)
  print(
  "\nRetrieved
  Pattern:"
)
print(retrieved)
  print( f"\nRetrieval
  Accuracy: "
  f"{accuracy:.2f}%"
)
print(
  f"Iterations to Converge: "
  f"{iterations}"
```

```
)
print(
  f"Initial Energy: "
  f"{energy_history[0]:.2f}"
)
print(
  f"Final Energy: "
  f"{energy_history[-
  1]:.2f}"
)


  display_res
    ult(
    noisy_inp
    ut,
    retrieved
    ,
  f"Noise = {noise}%"
)



plt.figure(figsiz
e=(7, 4))
plt.plot(
  energy_history,
  marker="o"
)
plt.xlabel(
  "Iteration"
)
```

```
  plt.ylabel(
    "Energy"
)
plt.title(
  "Energy Minimization"
)
plt.grid(True) plt.show()



elif choice == "2":
print("\n")
print("=" * 60)
print(" ENTER YOUR OWN 5×5 PATTERN")
print("=" *
60) print(
  "\nEnter 5 rows."
)
print(

  "Use 1 for ON and -1 for OFF."
)
print(
  "Example: 1 1 -1 -1 1"
)

user_pattern = []
valid_input = True
  for row_number in
  range(5): row = input(
    f"Row {row_number + 1}: "
```

```
)
try:
    values
      = [
      int(x)
    for x in row.split()
  ]
    if len(values) != 5:
    print(
      "\nEach row must
      contain " "exactly 5
      values."
    )
    valid_input =
    False break
  if not all(
    x in [-1, 1]
    for x in values
  ):
    print(

      "\nOnly -1 and 1 are allowed."
    )
    valid_input =
    False break
    user_pattern.appe
    nd(
      values
  )
 except:
   print(
```

```
      "\nInvalid input."
    )
    valid_input = False break

  if not
  valid_input:
  continue

  user_pattern = np.array(
    user_pattern
)

print("\nYour input pattern:") print(user_pattern)
  retrieved, energy_history,
    iterations = ( retrieve_pattern(
    user_pattern
  )
)

print(
  "\nHopfield Retrieved Pattern:"
)
print(retrieved) print(
  f"\nIterations to
  Converge: "
  f"{iterations}"
)
print(
  f"Initial Energy: "
  f"{energy_history[0]:.2f}"
)
```

```
    print(
      f"Final Energy: "
      f"{energy_history[-
      1]:.2f}"
    )


      display_result(
        user_pattern,
        retrieved,
      "Student Input → Hopfield Retrieval"
    )
plt.figure(figsize=(7, 4)) plt.plot(
      energy_history,
      marker="o"
    )
    plt.xlabel(

      "Iteration"
    )
      plt.ylabel(
        "Energy"
    )
    plt.title(
      "Energy During Retrieval"
    )
    plt.grid(True)
  plt.show() elif
  choice == "3":
    print("\nStored Memories:")
      for name, pattern in
```

```
    patterns.items(): display_pattern(
      pattern,
      f"Stored Pattern {name}"
    )
  elif choice == "4":
  print(
    "\nThank you for using the "
    "Hopfield Neural Network!"
  )
break
  else:
  print(
  "\nPlease enter 1, 2, 3 or 4."
  )
```

Output:-

```
HOPFIELD NEURAL NETWORK
INTERACTIVE LEARNING SYSTEM
============================================================

The network has learned these patterns:

Pattern A
[[-1  1  1  1 -1]
 [ 1 -1 -1 -1  1]
 [ 1  1  1  1  1]
 [ 1 -1 -1 -1  1]
 [ 1 -1 -1 -1  1]]

Pattern B
[[ 1  1  1  1 -1]
 [ 1 -1 -1 -1  1]
 [ 1  1  1  1 -1]
 [ 1 -1 -1 -1  1]
 [ 1  1  1  1 -1]]

Pattern C
[[-1  1  1  1  1]
 [ 1 -1 -1 -1 -1]
 [ 1 -1 -1 -1 -1]
 [ 1 -1 -1 -1 -1]
 [-1  1  1  1  1]]

------------------------------------------------
NETWORK INFORMATION
------------------------------------------------
```

```
Number of stored patterns : 3
Image size : 5 x 5
Number of neurons : 25
Weight matrix : (25, 25)


============================================================
 MAIN MENU
============================================================
1. Test a stored pattern with noise
2. Enter your own 5x5 pattern
3. Display stored patterns
4. Exit

Enter your choice (1-4): 2


============================================================
 ENTER YOUR OWN 5x5 PATTERN
============================================================

Enter 5 rows.
Use 1 for ON and -1 for OFF.
Example: 1 1 -1 -1 1
Row 1: 1   -1    1    -1   1
Row 2: 1       1        1          -1

Each row must contain exactly 5 values.
```

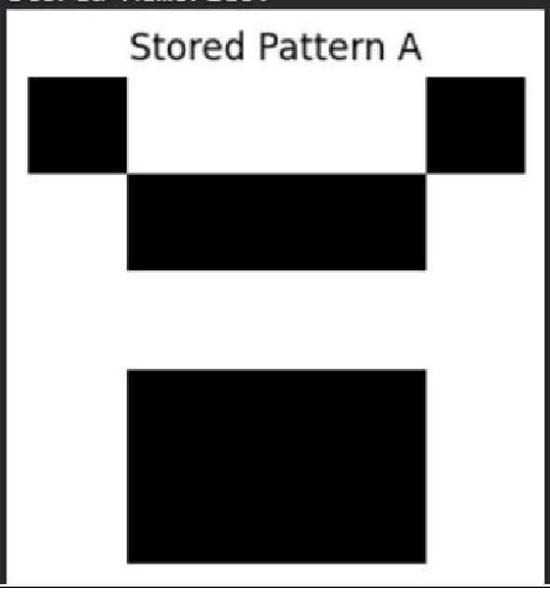
```
Each row must contain exactly 5 values.

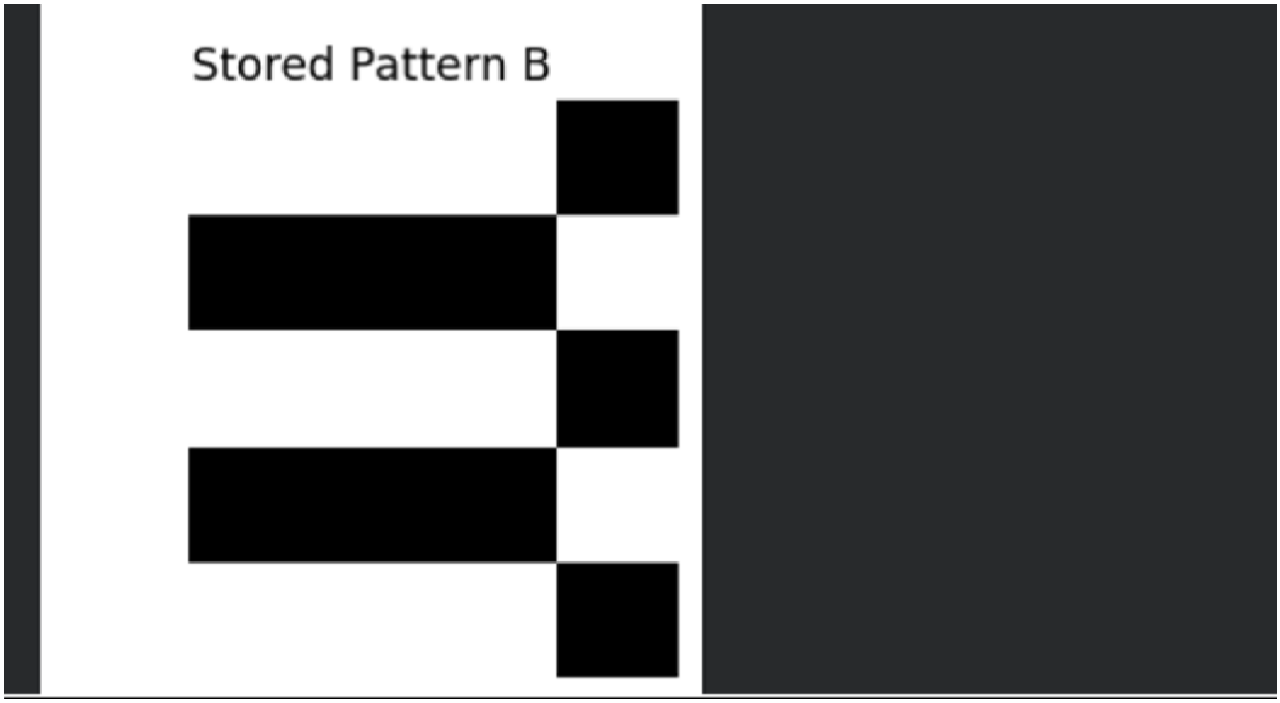

============================================================
 MAIN MENU
============================================================
1. Test a stored pattern with noise
2. Enter your own 5×5 pattern
3. Display stored patterns
4. Exit

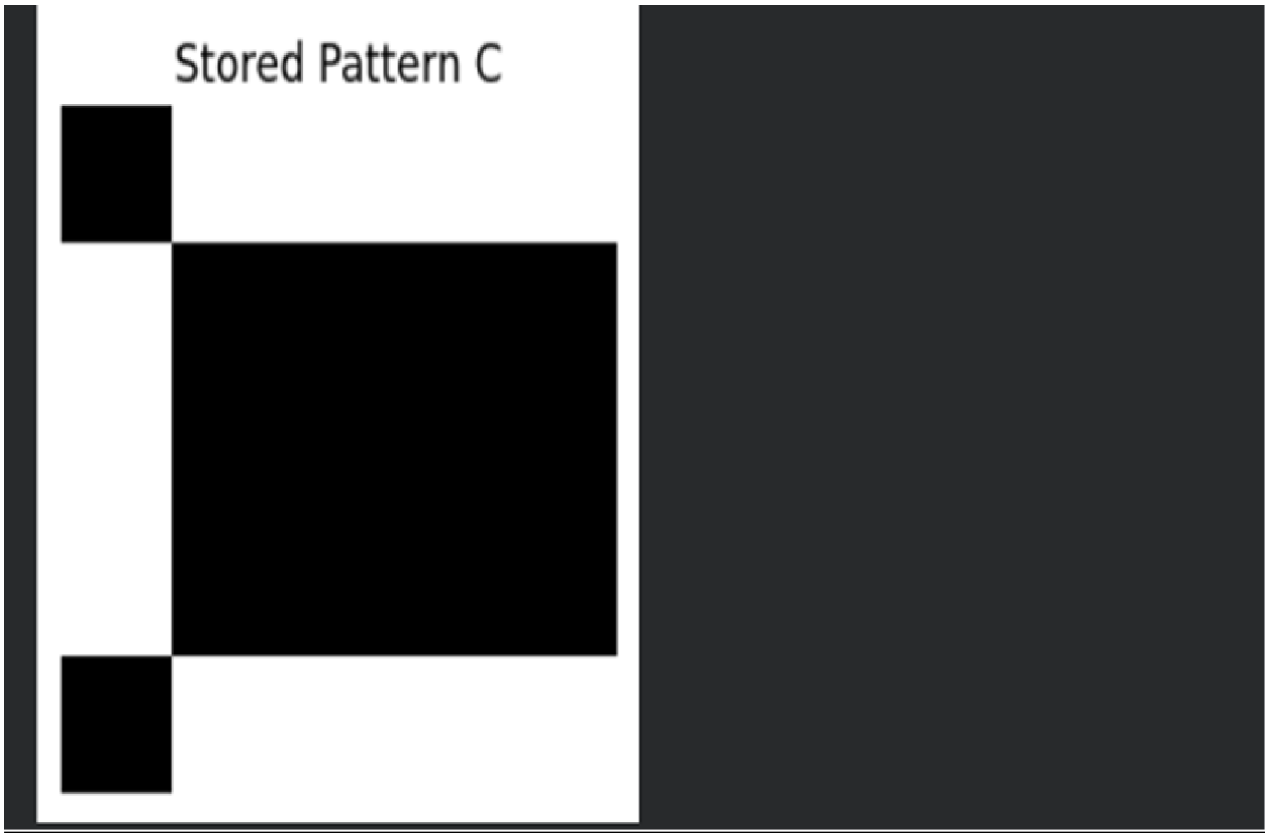
Enter your choice (1-4): 3
```

```
Enter your choice (1-4): 3

Stored Memories:
```




Stored Pattern B


Stored Pattern C

```
==========================================================

 MAIN MENU

==========================================================

1. Test a stored pattern with noise
2. Enter your own 5x5 pattern
3. Display stored patterns
4. Exit

Enter your choice (1-4): 4

Thank you for using the Hopfield Neural Network!
```

EXPERIMENTAL ANALYSIS OF A HOPFIELD NEURAL NETWORK USING MNIST

Source Code:

```
import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist


(x_train, y_train), (x_test, y_test) =
mnist.load_data() print("MNIST DATASET")
print("-" * 50)

print("Training images :",
x_train.shape) print("Testing
images :", x_test.shape)


plt.figure(figsize
=(10, 4)) for i in
range(10):
  plt.subplot(2, 5, i +
  1) plt.imshow(
    x_train[i],
    cmap="gray"
  )
  plt.title(
    f"Digit: {y_train[i]}"
  )
  plt.axis("off"
)
plt.tight_layout
() plt.show()


digit_to_store =
5 index =
```

```
np.where(
  y_train == digit_to_store
)[0][0]
original_image = x_train[index]
print("\nSelected digit:",
digit_to_store) print("Image
shape:", original_image.shape)

from PIL import Image
  image_pil =
  Image.fromarray(
    original_image
)
  image_pil =
    image_pil.resize( (14,
    14)
)
  small_image =
    np.array( image_pil
)
print("\nResized image shape:")
print(small_image.shape)

threshold = 80 binary_pattern =
np.where(
  small_image > threshold, 1,
  -1
)
plt.figure(figsiz
e=(4, 4))
```

```
plt.imshow(
  binary_patt
  ern,
  cmap="gray"
  , vmin=-1,

  vmax=1
)
plt.title(
  "Binary MNIST Pattern"
)
plt.axis("off")
plt.show()

stored_pattern = binary_pattern.flatten()
number_of_neurons = len(
  stored_pattern
)
print("\nNumber of Hopfield neurons:")
print(number_of_neurons)

  W = np.outer(
    stored_patter
    n,
    stored_patter
    n
)

  np.fill_diag
    onal( W,
  0
```

```
)


W = W / number_of_neurons
print("\nWeight Matrix Shape:")
print(W.shape)


  def
    calculate_energy(st
    ate): state =
    state.flatten()
    energy = -0.5 *
    np.dot(
      state,
    np.dot(W,
    state)
  )
  return energy


  def add_noise(
    pattern,
  noise_percentage
):
  noisy_pattern = pattern.copy() total_pixels
  = len(
    noisy_pattern
  )
    number_of_noisy_pixels
    = int( total_pixels *
    noise_percentage /
    100
  )
```

```
      indices = np.random.choice(
      total_pixels,
      number_of_noisy_pixels,
      replace=False
    )
    noisy_pattern[indices
    ] *= -1 return
    noisy_pattern
    def
      update_network(stat
      e):   new_state   =
      state.copy()  for  i
      in range(
      len(state)
    ):
        activation =
        np.dot( W[i],
        new_state
      )
        if activation >=
        0: new_state[i] =
        1
      else:
        new_state[i] = -1
    return new_state

    def retrieve_pattern(
      noisy_pattern,
      max_iterations=20
  ):
    state = noisy_pattern.copy()
```

```
energy_history = []


  energy_history.append(
    calculate_energy(state)
)
  for iteration in
    range(
    max_iterations
):
  new_state = update_network(
    state
  )


    energy_history.append(
    calculate_energy(
      new_state
    )
  )


    if
    np.array_equa
    l( new_state,
    state
  ):
    print(
      "\nNetwork converged at " f"iteration
      {iteration + 1}"
    )
    break
  state = new_state return
(
```

```
    state,
    energy_histor
    y
  )


  def
    calculate_accuracy(
    original,
  retrieved
):
  correct_pixels = np.sum(
    original == retrieved
  )
    total_pixels =
      len(
      original
  )
  accuracy = (
    correct_pix
    els /
    total_pixel
    s
  ) * 100
  return accuracy


noise_percentage = 20
noisy_pattern =
add_noise(
  stored_pattern
  ,
  noise_percenta
```

```
  ge
)
  retrieved_pattern, energy_history = (
    retrieve_pattern(
    noisy_pattern
  )
)


  accuracy =
    calculate_accuracy(
    stored_pattern,
  retrieved_pattern
)
print("\n")
print("=" * 60)
print("RETRIEVAL RESULTS")
print("=" * 60)
  print( "Noise
  Level :",
  noise_percentage, "%"
)
print(
  "Retrieval
  Accuracy :",
  round(accuracy,
  2), "%"
)
print(
  "Initial Energy :", round(
    energy_history[0], 2
```

```
  )
)
print(
  "Final Energy
  :", round(
    energy_history[-1], 2
  )
)


  noisy_image =
    noisy_pattern.reshape( 14,
  14
)
  retrieved_image =
    retrieved_pattern.reshape( 14,
  14
)


  plt.figure(
    figsize=(12
    , 4)
)
  plt.subpl
    ot( 1,
  3,
  1
)
  plt.imshow(
    binary_patt
```

```
    ern,
    cmap="gray"
    , vmin=-1,
    vmax=1
)
plt.title(
  "Original Stored Pattern"
)
plt.axis("off")
plt.subplot(
  1,
  3,
  2
)
  plt.imshow(
    noisy_image,
    cmap="gray",
    vmin=-1,
    vmax=1
)
plt.title(
  f"Noisy Input ({noise_percentage}%)"
)
plt.axis("off")
plt.subplot(
  1,
  3,
  3
)
```

```
plt.imshow(
  retrieved_ima
  ge,
  cmap="gray",
  vmin=-1,
  vmax=1
)
plt.title(
  "Retrieved Pattern"
)
plt.axis("off")
plt.tight_layout
() plt.show()

plt.figure(
  figsize=(7, 4)
)
  plt.plot(
    range(
    len(energy_hist
    ory)
  ),
  energy_history
  , marker="o"
)
plt.xlabel(
  "Iteration"
)
  plt.ylabel(
    "Energy"
```

```
)
plt.title(
  "Hopfield Energy Minimization"
)
  plt.grid
    ( True
)
plt.show()


  noise_levels
    = [ 0,
  10,
  20,
  30,
  40,
  50
]
accuracies = []
  for noise in
    noise_levels: noisy
    = add_noise(
    stored_pattern,
    noise
  )
    retrieved, energies = (
    retrieve_pattern(
      noisy
    )
  )
```

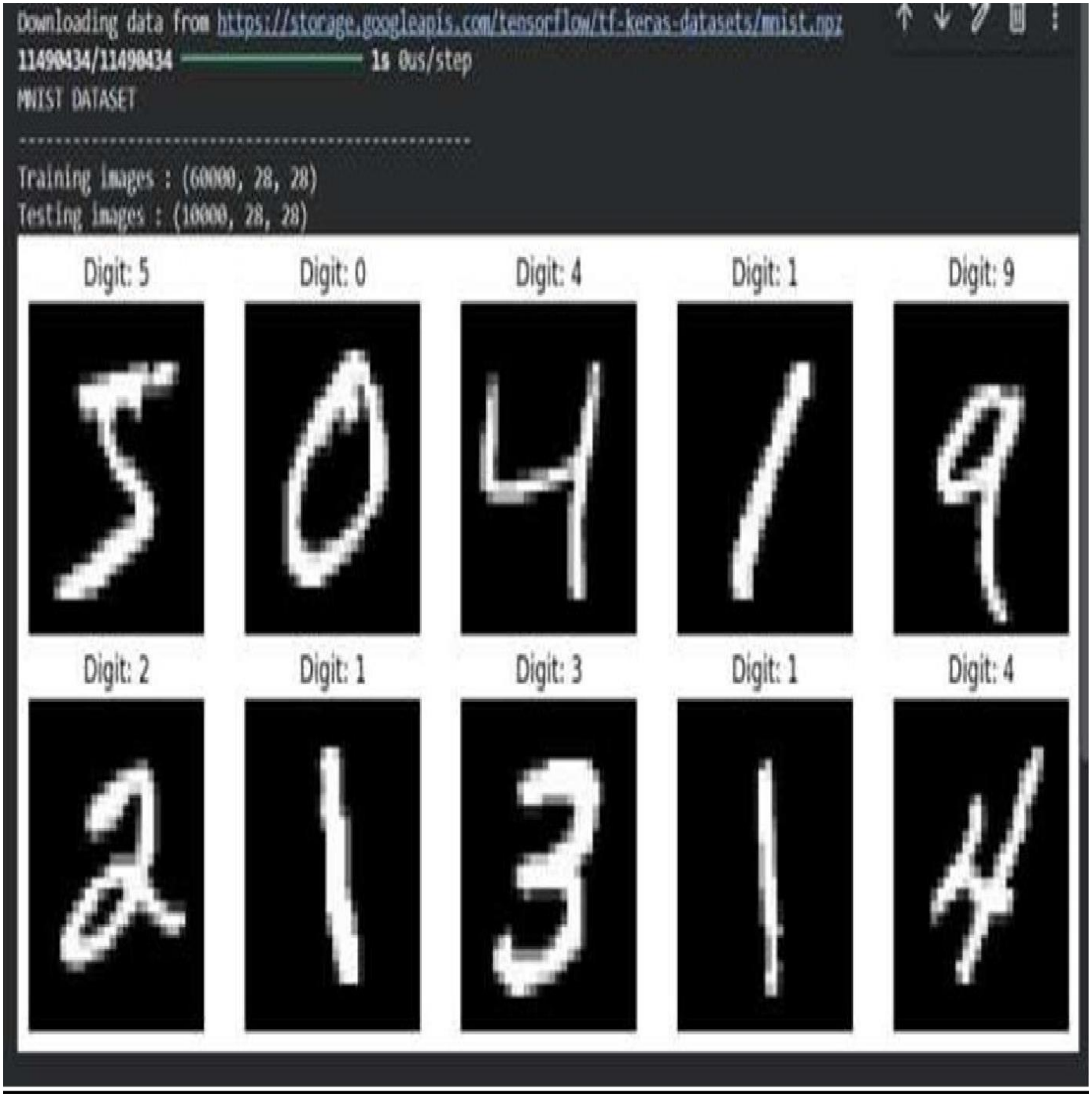
```
    acc = calculate_accuracy(
      stored_pattern,
    retrieved
  )
    accuracies.appe
      nd( acc
  )


print("\n")
print("=" * 60)
print("NOISE TOLERANCE ANALYSIS")
print("=" * 60) print(
  "\nNoise Level (%) Retrieval Accuracy (%)"
)
print(
  "-" * 45)
  for noise, accuracy_value in
    zip( noise_levels,
  accuracies
):
  print(
    f"{noise:^16} "
    f"{accuracy_value:>10.2f}"
  )


  plt.figure(
    figsize=(7
    , 4)
)
```

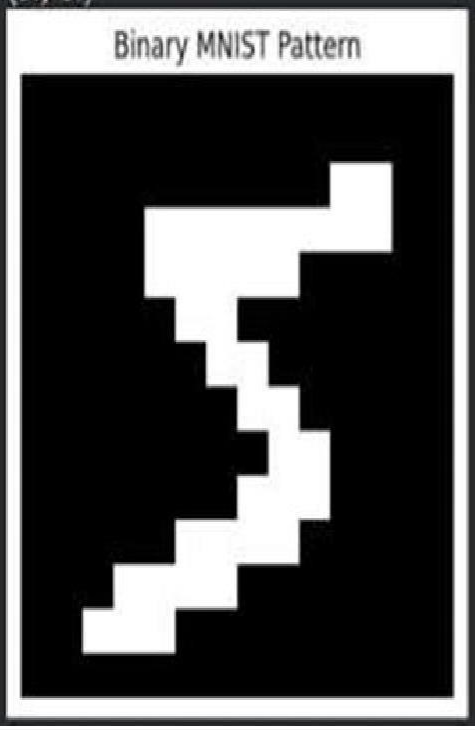
```
plt.plot(
  noise_levels,
  accuracies,
  marker="o"
)
plt.xlabel(
  "Noise Level (%)"
)
plt.ylabel(
  "Retrieval Accuracy (%)"
)
plt.title(
  "Hopfield Network Noise Tolerance"
)
plt.grid(
  True
)
plt.show()
print("\n")
print("=" * 60)
print("EXPERIMENT COMPLETED")
print("=" * 60)
```

OUTPUT:-


Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
MNIST DATASET
Training Images : (60000, 28, 28)
Testing Images : (10000, 28, 28)
Digit: 5
Digit: 0
Digit: 4
Digit: 1
Digit: 9
Digit: 2
Digit: 1
Digit: 3
Digit: 1
Digit: 4

```
Selected digit: 5
Image shape: (28, 28)

Resized image shape:
(14, 14)
```

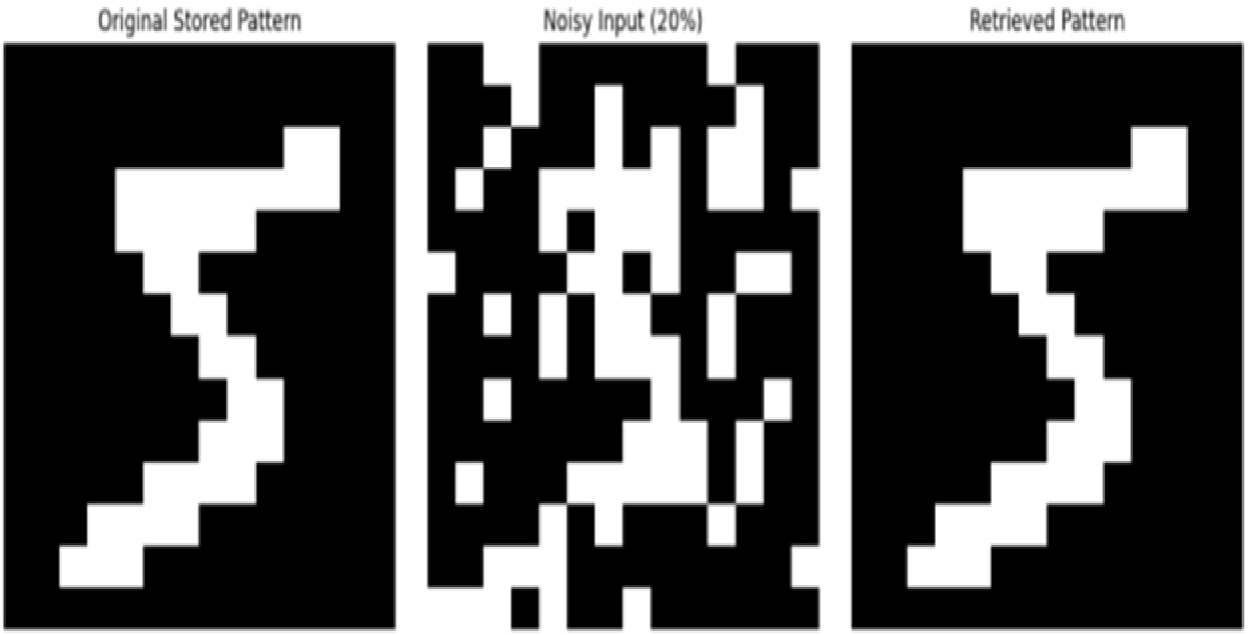


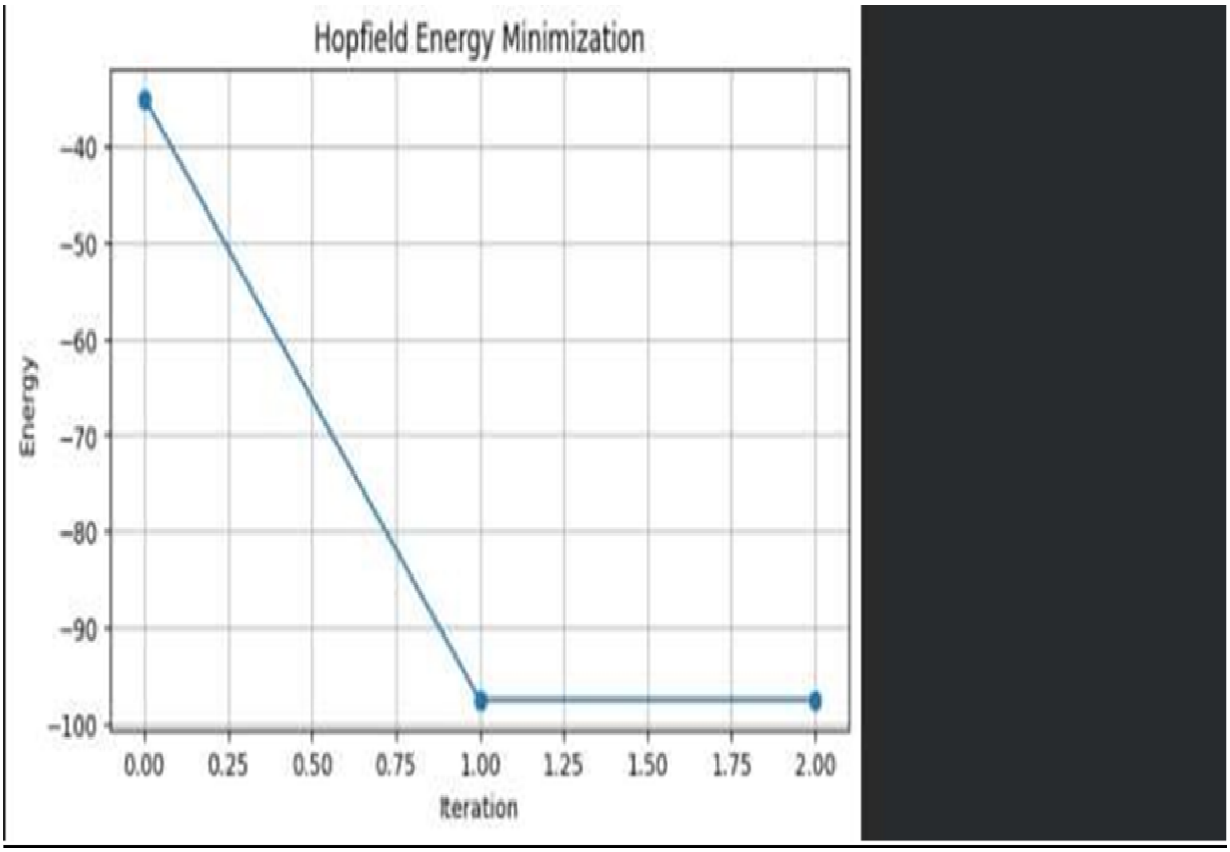

```
Number of Hopfield neurons:
196

Weight Matrix Shape:
(196, 196)

Network converged at iteration 2


============================================================
RETRIEVAL RESULTS
============================================================
Noise Level : 20 %
Retrieval Accuracy : 100.0 %
Initial Energy : -35.02
Final Energy : -97.5
```


Original Stored Pattern
Noisy Input (20%)
Retrieved Pattern


Hopfield Energy Minimization
Energy
-40
-50
-60
-70
-80
-90
-100
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
Iteration

```
Network converged at iteration 1

Network converged at iteration 2

Network converged at iteration 2

Network converged at iteration 2

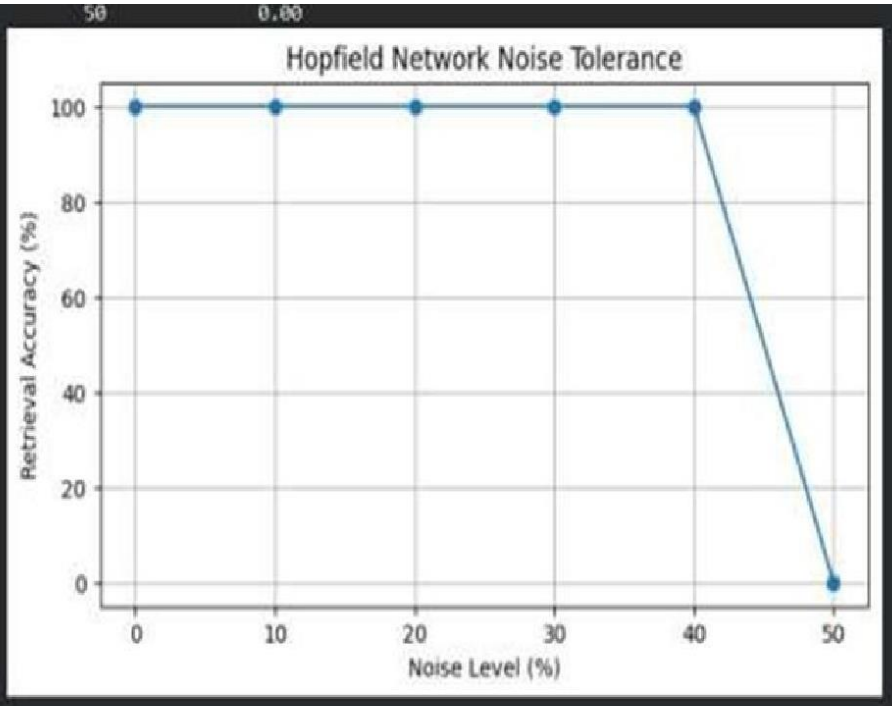
Network converged at iteration 2

Network converged at iteration 2


============================================================
NOISE TOLERANCE ANALYSIS
============================================================

Noise Level (%) Retrieval Accuracy (%)
------------------------------------------------
       0              100.00
      10              100.00
      20              100.00
      30              100.00
      40              100.00
      50                0.00
```

```
      50                0.00
```




```
============================================================
EXPERIMENT COMPLETED
============================================================
```

PRACTICAL :- 7

Practical-7

Aim :-Fuzzy Logic Based Air Conditioner Control System

Problem Statement-1

Design a Fuzzy Inference System that controls air conditioner fan speed based on room temperature and humidity.

Source Code:-

```
# Fuzzy Logic Based Air Conditioner Control System
# Step 1: Install required library
!pip install scikit-fuzzy -q
# Step 2: Import required libraries
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt
# Step 3: Define Input and Output Variables
# Temperature: 0 to 50 degree Celsius
temperature = ctrl.Antecedent(np.arange(0, 51, 1), 'temperature')
# Humidity: 0 to 100 percent
humidity = ctrl.Antecedent(np.arange(0, 101, 1), 'humidity')
# Fan Speed: 0 to 100 percent
fan_speed = ctrl.Consequent(np.arange(0, 101, 1), 'fan_speed')

# Step 4: Define Membership Functions
# ---------- Temperature Membership Functions ----------
temperature['cold'] = fuzz.trimf(
temperature.universe, [0, 0, 20]
)

temperature['comfortable'] = fuzz.trimf(
temperature.universe, [15, 25, 35]
)

temperature['hot'] = fuzz.trimf(
temperature.universe, [30, 50, 50]
)
```

```
# ---------- Humidity Membership Functions ----------
humidity['dry'] = fuzz.trimf(
humidity.universe, [0, 0, 40]
)

humidity['normal'] = fuzz.trimf(
humidity.universe, [30, 50, 70]
)

humidity['humid'] = fuzz.trimf(
humidity.universe, [60, 100, 100]
)

# ---------- Fan Speed Membership Functions ----------
fan_speed['low'] = fuzz.trimf(

fan_speed.universe, [0, 0, 40]
)

fan_speed['medium'] = fuzz.trimf(
fan_speed.universe, [30, 50, 70]
)

fan_speed['high'] = fuzz.trimf(
fan_speed.universe, [60, 100, 100]
)
# Step 5: Display Membership Functions
temperature.view()
plt.title("Temperature Membership Functions")
plt.show()

humidity.view()
plt.title("Humidity Membership Functions")
plt.show()
fan_speed.view()
plt.title("Fan Speed Membership Functions")
plt.show()
# Step 6: Define Fuzzy Rules
# Rule 1: Cold + Dry -> Low Fan Speed

rule1 = ctrl.Rule(
temperature['cold'] & humidity['dry'],
```

```
fan_speed['low']
)

# Rule 2: Cold + Normal -> Low Fan Speed
rule2 = ctrl.Rule(
temperature['cold'] & humidity['normal'],
fan_speed['low']
)

# Rule 3: Cold + Humid -> Medium Fan Speed
rule3 = ctrl.Rule(
temperature['cold'] & humidity['humid'],
fan_speed['medium']
)

# Rule 4: Comfortable + Dry -> Medium Fan Speed
rule4 = ctrl.Rule(

temperature['comfortable'] & humidity['dry'],
fan_speed['medium']
)
# Rule 5: Comfortable + Normal -> Medium Fan Speed
rule5 = ctrl.Rule(
temperature['comfortable'] & humidity['normal'],
fan_speed['medium']
)

# Rule 6: Comfortable + Humid -> High Fan Speed
rule6 = ctrl.Rule(
temperature['comfortable'] & humidity['humid'],
fan_speed['high']
)
# Rule 7: Hot + Dry -> High Fan Speed
rule7 = ctrl.Rule(
temperature['hot'] & humidity['dry'],
fan_speed['high']
)
# Rule 8: Hot + Normal -> High Fan Speed
rule8 = ctrl.Rule(
temperature['hot'] & humidity['normal'],
fan_speed['high']
)
# Rule 9: Hot + Humid -> High Fan Speed
```

```
rule9 = ctrl.Rule(
temperature['hot'] & humidity['humid'],
fan_speed['high']
)
# Step 7: Create Fuzzy Control System
fan_control = ctrl.ControlSystem([
rule1, rule2, rule3,
rule4, rule5, rule6,
rule7, rule8, rule9
])

fan_simulation = ctrl.ControlSystemSimulation(fan_control)
# Step 8: Take Input from User
temp_input = float(input("Enter room temperature (°C): "))
humidity_input = float(input("Enter room humidity (%): "))

# Step 9: Validate Input
if not (0 <= temp_input <= 50):

    raise ValueError("Temperature must be between 0 and 50 °C.")

if not (0 <= humidity_input <= 100):
    raise ValueError("Humidity must be between 0 and 100 %.")

# Step 10: Pass Inputs to Fuzzy System

fan_simulation.input['temperature'] = temp_input
fan_simulation.input['humidity'] = humidity_input

# Step 11: Perform Fuzzy Inference and Defuzzification

fan_simulation.compute()

output_speed = fan_simulation.output['fan_speed']

# Step 12: Display Result
print("\n========================================")

print(" AIR CONDITIONER CONTROL SYSTEM")
print("========================================")
print(f"Room Temperature : {temp_input:.2f} °C")
print(f"Room Humidity : {humidity_input:.2f} %")
print(f"Fan Speed : {output_speed:.2f} %")
print("========================================")
```

```
# Step 13: Classify Fan Speed

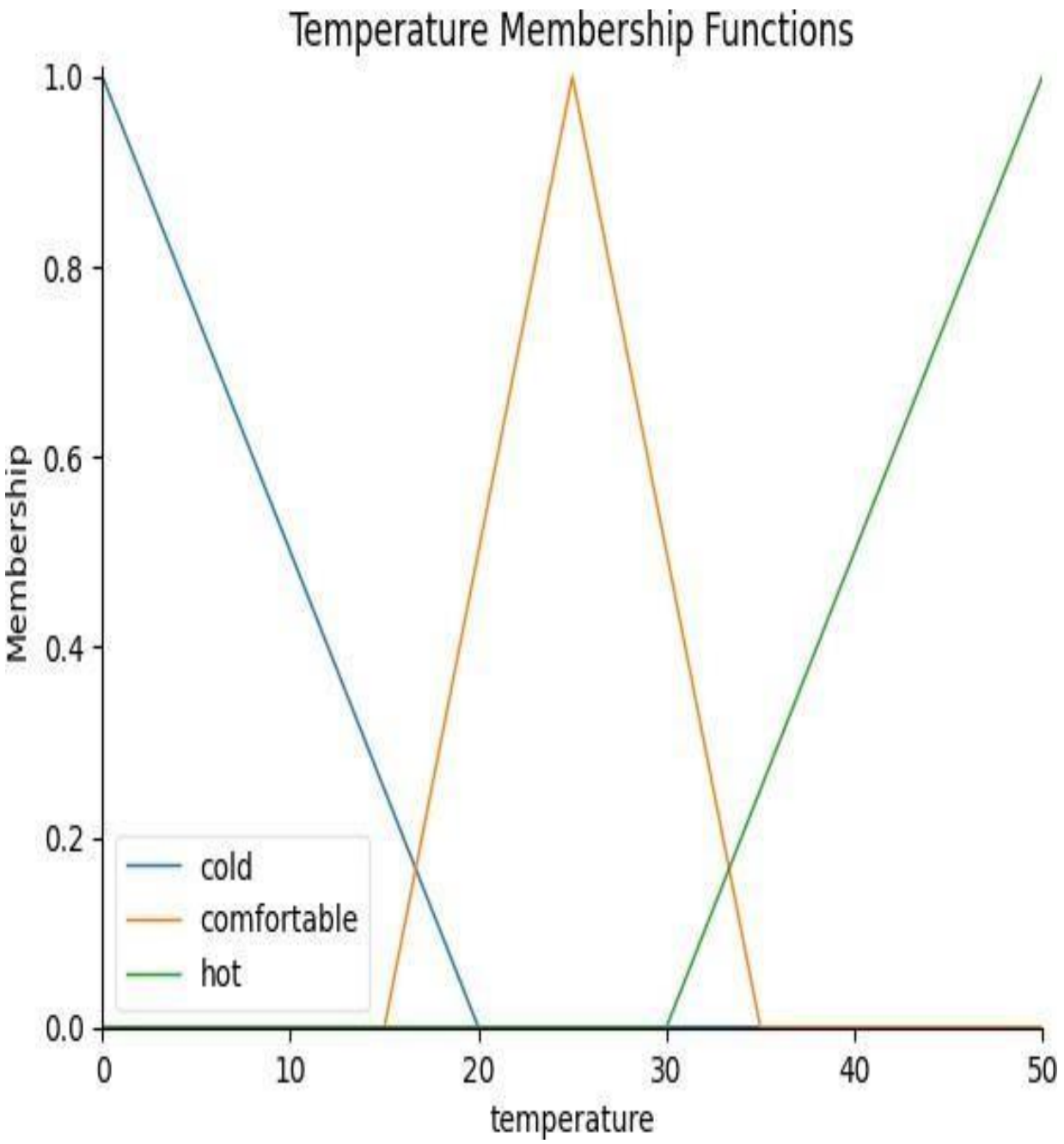
if output_speed < 40:
    speed_category = "LOW"
elif output_speed < 70:
    speed_category = "MEDIUM"
else:
    speed_category = "HIGH"

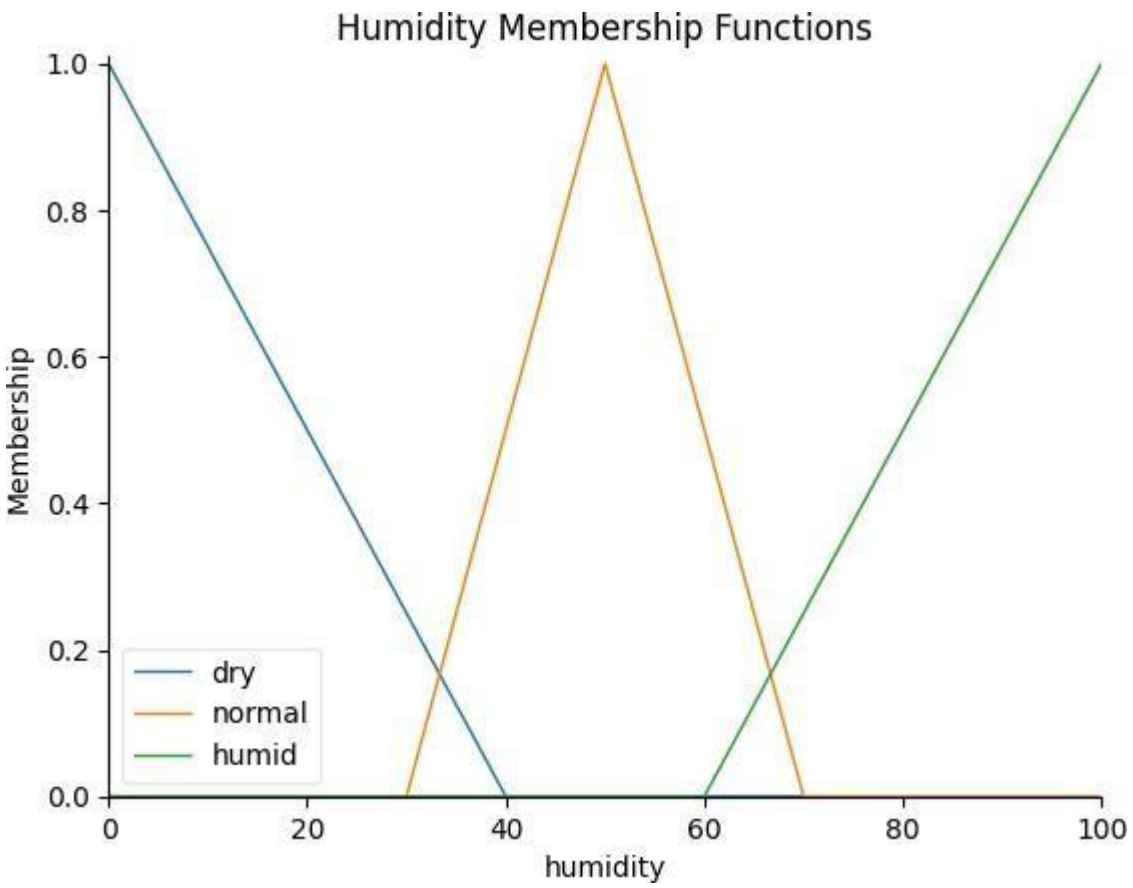
print(f"Fan Speed Level : {speed_category}")

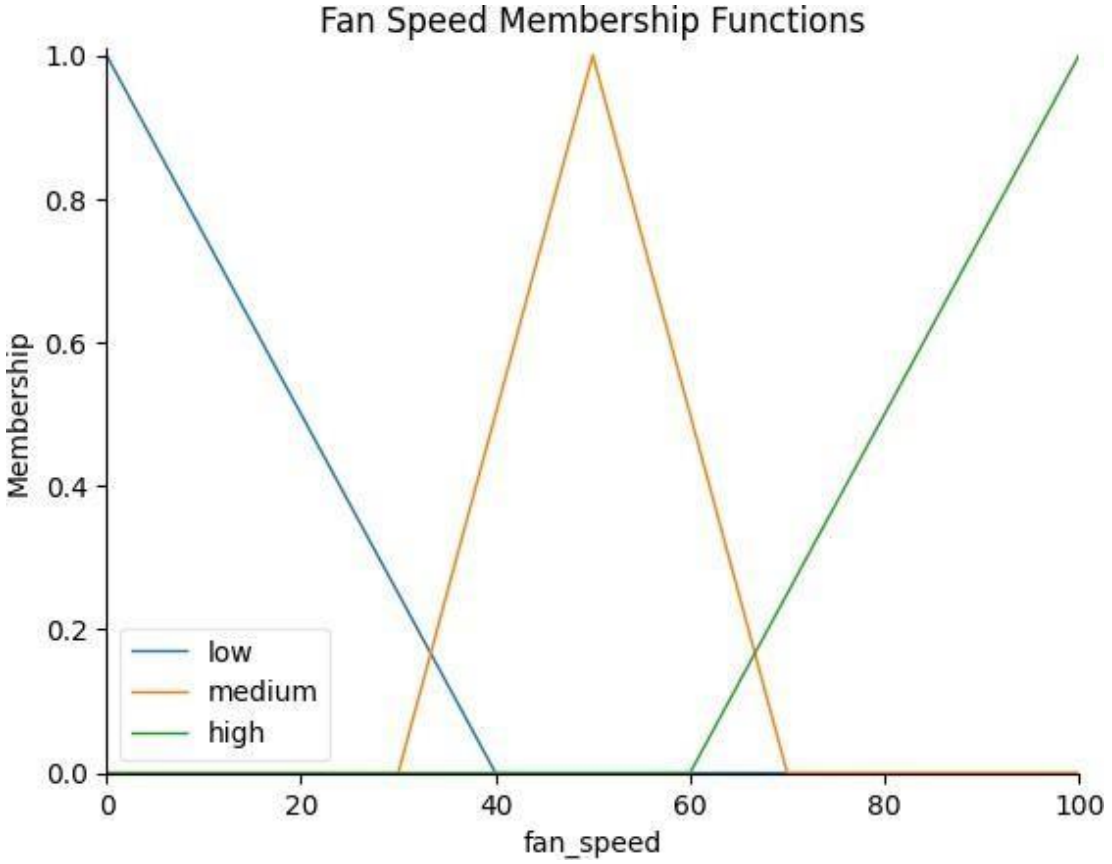
# Step 14: Display Defuzzification Result

fan_speed.view(sim=fan_simulation)
plt.title("Defuzzified Fan Speed")
plt.show()
```

OUTPUT:-


Temperature Membership Functions
1.0
0.8
0.6
0.4
0.2
0.0
Membership
cold
comfortable
hot
0
10
20
30
40
50
temperature


Humidity Membership Functions
Membership
humidity
dry
normal
humid
0.0
0.2
0.4
0.6
0.8
1.0
0
20
40
60
80
100


Fan Speed Membership Functions
Membership
fan_speed
low
medium
high
0.0
0.2
0.4
0.6
0.8
1.0
0
20
40
60
80
100

Enter room temperature (°C): 35
Enter room humidity (%): 74

==========

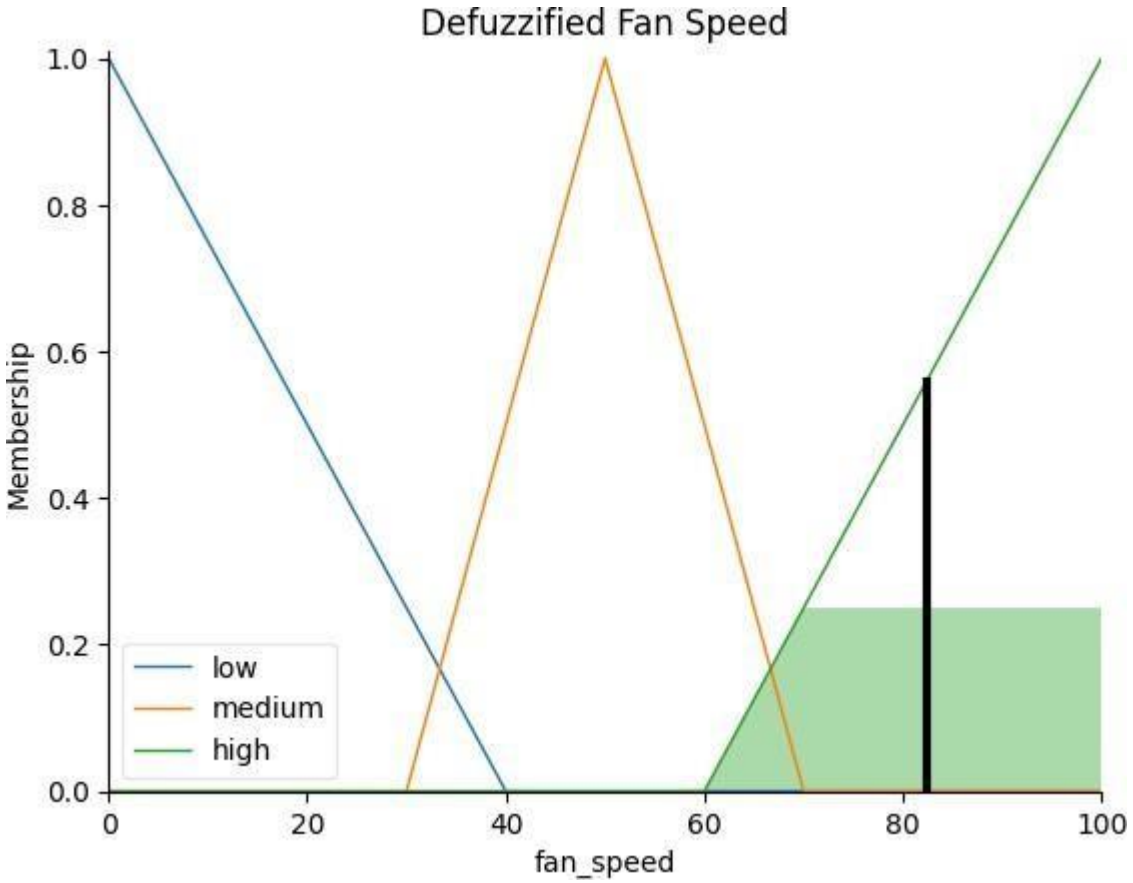
AIR CONDITIONER CONTROL SYSTEM

==========

Room Temperature : 35.00 °C
Room Humidity : 74.00 %
Fan Speed : 82.38 %

==========

Fan Speed Level : HIGH



PRACTICAL :- 8

Practical 08

Aim:- Fuzzy Expert System for Medical Diagnosis

Problem Statement-1

Develop a fuzzy expert system to estimate the risk level of diabetes based on blood sugar level, BMI and age.

Objectives

- **Apply fuzzy reasoning.**
- **Create rule base.**
- **Compare crisp and fuzzy decisions.**

Source Code:-

```
# ============================================================
# FUZZY EXPERT SYSTEM FOR MEDICAL DIAGNOSIS
# Diabetes Risk Estimation using Blood Sugar, BMI and Age
# ============================================================


# ============================================================
# Step 1: Install Required Library
# ============================================================

!pip install scikit-fuzzy -q


# ============================================================
# Step 2: Import Required Libraries
# ============================================================

import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt


# ============================================================
# Step 3: Define Input and Output Variables
# ============================================================

# Blood Sugar: 50 to 300 mg/dL
```

```
blood_sugar = ctrl.Antecedent(
    np.arange(50, 301, 1),
    'blood_sugar'
)

# BMI: 10 to 50
bmi = ctrl.Antecedent(
    np.arange(10, 51, 1),
    'bmi'
)

# Age: 10 to 100 years
age = ctrl.Antecedent(
    np.arange(10, 101, 1),
    'age'
)

# Diabetes Risk: 0 to 100 %
diabetes_risk = ctrl.Consequent(
    np.arange(0, 101, 1),
    'diabetes_risk'
)


# ============================================================
# Step 4: Define Membership Functions
# ============================================================

# ------------------------------------------------------------
# Blood Sugar Membership Functions
# ------------------------------------------------------------

blood_sugar['low'] = fuzz.trimf(
    blood_sugar.universe,
    [50, 50, 100]
)

blood_sugar['normal'] = fuzz.trimf(
    blood_sugar.universe,
    [80, 100, 125]
)

blood_sugar['high'] = fuzz.trimf(
    blood_sugar.universe,
```

```
    [110, 160, 220]
)




blood_sugar['very_high'] = fuzz.trimf(
    blood_sugar.universe,
    [180, 300, 300]
)


# ----------------------------------------------------------------------------------------------------
# BMI Membership Functions
# ----------------------------------------------------------------------------------------------------

bmi['underweight'] = fuzz.trimf(
    bmi.universe,
    [10, 10, 18.5]
)

bmi['normal'] = fuzz.trimf(
    bmi.universe,
    [17, 22, 25]
)

bmi['overweight'] = fuzz.trimf(
    bmi.universe,
    [23, 27, 30]
)

bmi['obese'] = fuzz.trimf(
    bmi.universe,
    [28, 40, 50]
)


# ----------------------------------------------------------------------------------------------------
# Age Membership Functions
# ----------------------------------------------------------------------------------------------------

age['young'] = fuzz.trimf(
```

```
    age.universe,
    [10, 10, 30]
)

age['middle'] = fuzz.trimf(
    age.universe,
    [25, 45, 60]
)

age['old'] = fuzz.trimf(
    age.universe,
    [50, 75, 100]
)


# ----------------------------------------------------------------------------------------------------------------
# Diabetes Risk Membership Functions
# ----------------------------------------------------------------------------------------------------------------

diabetes_risk['low'] = fuzz.trimf(
    diabetes_risk.universe,
    [0, 0, 40]
)

diabetes_risk['medium'] = fuzz.trimf(
    diabetes_risk.universe,
    [30, 50, 70]
)

diabetes_risk['high'] = fuzz.trimf(
    diabetes_risk.universe,
    [60, 100, 100]
)


# ============================================================
# Step 5: Display Membership Functions
# ============================================================

blood_sugar.view()
plt.title("Blood Sugar Membership Functions")
plt.show()
```

```
bmi.view()
plt.title("BMI Membership Functions")
plt.show()

age.view()
plt.title("Age Membership Functions")
plt.show()
diabetes_risk.view()
plt.title("Diabetes Risk Membership Functions")
plt.show()


# ============================================================
# Step 6: Define Fuzzy Rules
# ============================================================

rules = []


# ============================================================
# BLOOD SUGAR = LOW
# ============================================================

rules.append(ctrl.Rule(
    blood_sugar['low'] &
    bmi['underweight'] &
    age['young'],
    diabetes_risk['low']
))

rules.append(ctrl.Rule(
    blood_sugar['low'] &
    bmi['underweight'] &
    age['middle'],
    diabetes_risk['low']
))

rules.append(ctrl.Rule(
    blood_sugar['low'] &
    bmi['underweight'] &
    age['old'],
    diabetes_risk['low']
))
```

```
rules.append(ctrl.Rule(
    blood_sugar['low'] &
    bmi['normal'] &
    age['young'],
    diabetes_risk['low']
))

rules.append(ctrl.Rule(
    blood_sugar['low'] &
    bmi['normal'] &
    age['middle'],
    diabetes_risk['low']
))

rules.append(ctrl.Rule(
    blood_sugar['low'] &
    bmi['normal'] &
    age['old'],
    diabetes_risk['medium']
))

rules.append(ctrl.Rule(
    blood_sugar['low'] &
    bmi['overweight'] &
    age['young'],
    diabetes_risk['medium']
))

rules.append(ctrl.Rule(
    blood_sugar['low'] &
    bmi['overweight'] &
    age['middle'],
    diabetes_risk['medium']
))

rules.append(ctrl.Rule(
    blood_sugar['low'] &
    bmi['overweight'] &
    age['old'],
    diabetes_risk['medium']
))

rules.append(ctrl.Rule(
```

```
        blood_sugar['low'] &
        bmi['obese'] &
        age['young'],
        diabetes_risk['medium']
    ))
    rules.append(ctrl.Rule(
        blood_sugar['low'] &
        bmi['obese'] &
        age['middle'],
        diabetes_risk['medium']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['low'] &
        bmi['obese'] &
        age['old'],
        diabetes_risk['high']
    ))


    # ============================================================
    # BLOOD SUGAR = NORMAL
    # ============================================================

    rules.append(ctrl.Rule(
        blood_sugar['normal'] &
        bmi['underweight'] &
        age['young'],
        diabetes_risk['low']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['normal'] &
        bmi['underweight'] &
        age['middle'],
        diabetes_risk['low']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['normal'] &
        bmi['underweight'] &
        age['old'],
        diabetes_risk['medium']
    ))
```

```
rules.append(ctrl.Rule(
    blood_sugar['normal'] &
    bmi['normal'] &
    age['young'],
    diabetes_risk['low']
))

rules.append(ctrl.Rule(
    blood_sugar['normal'] &
    bmi['normal'] &
    age['middle'],
    diabetes_risk['low']
))

rules.append(ctrl.Rule(
    blood_sugar['normal'] &
    bmi['normal'] &
    age['old'],
    diabetes_risk['medium']
))

rules.append(ctrl.Rule(
    blood_sugar['normal'] &
    bmi['overweight'] &
    age['young'],
    diabetes_risk['medium']
))

rules.append(ctrl.Rule(
    blood_sugar['normal'] &
    bmi['overweight'] &
    age['middle'],
    diabetes_risk['medium']
))

rules.append(ctrl.Rule(
    blood_sugar['normal'] &
    bmi['overweight'] &
    age['old'],
    diabetes_risk['high']
))

rules.append(ctrl.Rule(
```

```
    blood_sugar['normal'] &
    bmi['obese'] &
    age['young'],
    diabetes_risk['medium']
))

rules.append(ctrl.Rule(
    blood_sugar['normal'] &
    bmi['obese'] &
    age['middle'],
    diabetes_risk['high']
))

rules.append(ctrl.Rule(
    blood_sugar['normal'] &
    bmi['obese'] &
    age['old'],
    diabetes_risk['high']
))


# ============================================================
# BLOOD SUGAR = HIGH
# ============================================================

rules.append(ctrl.Rule(
    blood_sugar['high'] &
    bmi['underweight'] &
    age['young'],
    diabetes_risk['medium']
))

rules.append(ctrl.Rule(
    blood_sugar['high'] &
    bmi['underweight'] &
    age['middle'],
    diabetes_risk['high']
))

rules.append(ctrl.Rule(
    blood_sugar['high'] &
    bmi['underweight'] &
    age['old'],
```

```
        diabetes_risk['high']
    ))
    rules.append(ctrl.Rule(
        blood_sugar['high'] &
        bmi['normal'] &
        age['young'],
        diabetes_risk['medium']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['high'] &
        bmi['normal'] &
        age['middle'],
        diabetes_risk['high']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['high'] &
        bmi['normal'] &
        age['old'],
        diabetes_risk['high']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['high'] &
        bmi['overweight'] &
        age['young'],
        diabetes_risk['high']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['high'] &
        bmi['overweight'] &
        age['middle'],
        diabetes_risk['high']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['high'] &
        bmi['overweight'] &
        age['old'],
        diabetes_risk['high']
    ))
    rules.append(ctrl.Rule(
```

```
        blood_sugar['high'] &
        bmi['obese'] &
        age['young'],
        diabetes_risk['high']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['high'] &
        bmi['obese'] &
        age['middle'],
        diabetes_risk['high']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['high'] &
        bmi['obese'] &
        age['old'],
        diabetes_risk['high']
    ))


    # ============================================================
    # BLOOD SUGAR = VERY HIGH
    # ============================================================

    rules.append(ctrl.Rule(
        blood_sugar['very_high'] &
        bmi['underweight'] &
        age['young'],
        diabetes_risk['high']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['very_high'] &
        bmi['underweight'] &
        age['middle'],
        diabetes_risk['high']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['very_high'] &
        bmi['underweight'] &
```

```
        age['old'],
        diabetes_risk['high']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['very_high'] &
        bmi['normal'] &
        age['young'],
        diabetes_risk['high']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['very_high'] &
        bmi['normal'] &
        age['middle'],
        diabetes_risk['high']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['very_high'] &
        bmi['normal'] &
        age['old'],
        diabetes_risk['high']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['very_high'] &
        bmi['overweight'] &
        age['young'],
        diabetes_risk['high']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['very_high'] &
        bmi['overweight'] &
        age['middle'],
        diabetes_risk['high']
    ))

    rules.append(ctrl.Rule(
        blood_sugar['very_high'] &
        bmi['overweight'] &
        age['old'],
```

```
    diabetes_risk['high']
))

rules.append(ctrl.Rule(
    blood_sugar['very_high'] &
    bmi['obese'] &
    age['young'],
    diabetes_risk['high']
))

rules.append(ctrl.Rule(
    blood_sugar['very_high'] &
    bmi['obese'] &
    age['middle'],
    diabetes_risk['high']
))

rules.append(ctrl.Rule(
    blood_sugar['very_high'] &
    bmi['obese'] &
    age['old'],
    diabetes_risk['high']
))


# ============================================================
# Step 7: Create Fuzzy Control System
# ============================================================

diabetes_control = ctrl.ControlSystem(rules)

diabetes_simulation = ctrl.ControlSystemSimulation(
    diabetes_control
)


# ============================================================
# Step 8: Take Input from User
# ============================================================

blood_sugar_input = float(
    input("Enter blood sugar level (mg/dL): ")
)
```

```
bmi_input = float(
    input("Enter BMI: ")
)

age_input = float(
    input("Enter age (years): ")
)


# ============================================================
# Step 9: Validate Input
# ============================================================

if not (50 <= blood_sugar_input <= 300):
    raise ValueError(
        "Blood sugar must be between 50 and 300 mg/dL."
    )

if not (10 <= bmi_input <= 50):
    raise ValueError(
        "BMI must be between 10 and 50."
    )

if not (10 <= age_input <= 100):
    raise ValueError(
        "Age must be between 10 and 100 years."
    )


# ============================================================
# Step 10: Pass Inputs to Fuzzy System
# ============================================================

diabetes_simulation.input['blood_sugar'] = blood_sugar_input
diabetes_simulation.input['bmi'] = bmi_input
diabetes_simulation.input['age'] = age_input


# ============================================================
# Step 11: Perform Fuzzy Inference and Defuzzification
# ============================================================
```

```
diabetes_simulation.compute()

output_risk = diabetes_simulation.output['diabetes_risk']


# ============================================================
# Step 12: Display Fuzzy Result
# ============================================================

print("\n================================================")
print("         FUZZY DIABETES RISK SYSTEM")
print("================================================")

print(
    f"Blood Sugar      : {blood_sugar_input:.2f} mg/dL"
)

print(
    f"BMI              : {bmi_input:.2f}"
)

print(
    f"Age              : {age_input:.2f} years"
)

print(
    f"Fuzzy Risk Score : {output_risk:.2f} %"
)

print("================================================")


# ============================================================
# Step 13: Classify Fuzzy Risk
# ============================================================

if output_risk < 40:
    fuzzy_category = "LOW"

elif output_risk < 70:
    fuzzy_category = "MEDIUM"

else:
```

```
    fuzzy_category = "HIGH"


print(
    f"Fuzzy Risk Level : {fuzzy_category}"
)


# ============================================================
# Step 14: Crisp Decision
# ============================================================

# Simple threshold-based decision for comparison.
# This is an academic comparison and is not a clinical
# diagnostic method.

if blood_sugar_input < 100:
    crisp_category = "LOW"

elif blood_sugar_input < 126:
    crisp_category = "MEDIUM"

else:
    crisp_category = "HIGH"


# ============================================================
# Step 15: Display Crisp vs Fuzzy Decision
# ============================================================

print("\n================================================")
print("          CRISP VS FUZZY DECISION")
print("================================================")

print(
    f"Crisp Decision   : {crisp_category}"
)

print(
    f"Fuzzy Decision   : {fuzzy_category}"
)

print(
```

```
    f"Fuzzy Risk Score : {output_risk:.2f} %"
)

print("================================================")


# ============================================================
# Step 16: Compare the Results
# ============================================================

if crisp_category == fuzzy_category:

    print(
        "Result: Crisp and Fuzzy decisions are SAME."
    )

else:

    print(
        "Result: Crisp and Fuzzy decisions are DIFFERENT."
    )

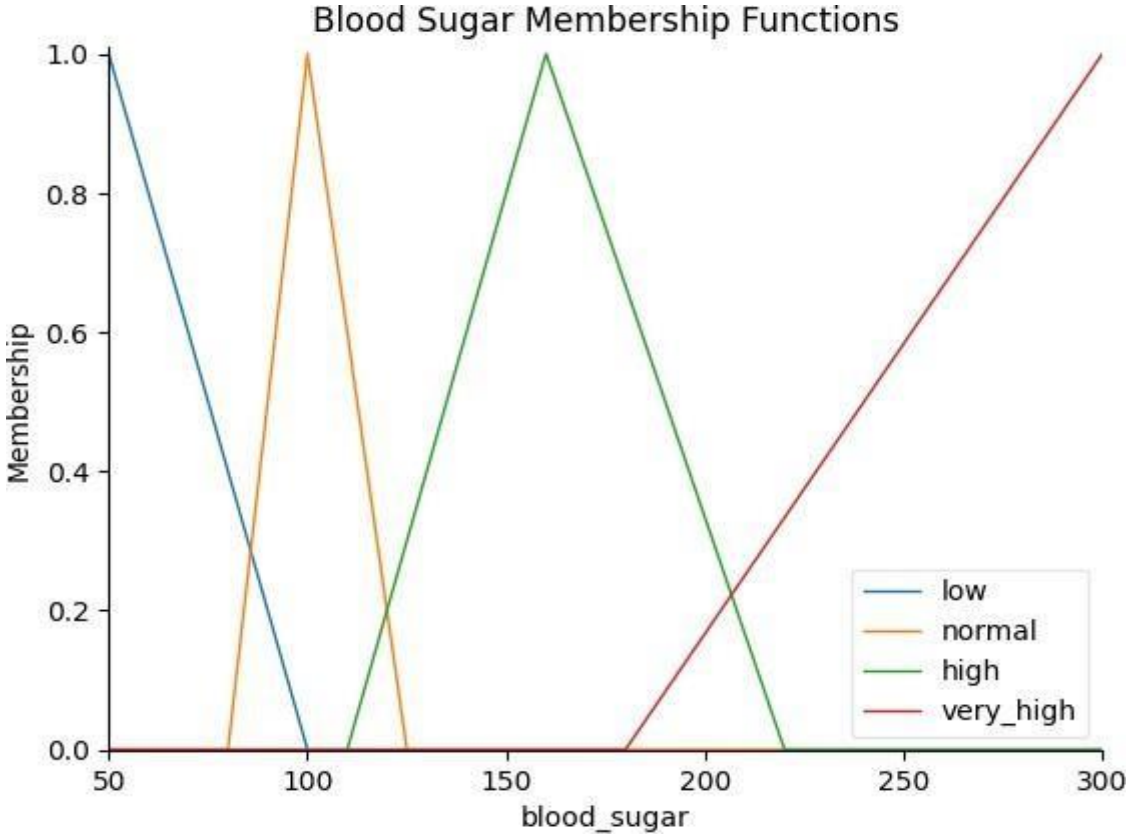
    print(
        "Fuzzy logic considers Blood Sugar, BMI and Age "
        "together using gradual membership values."
    )

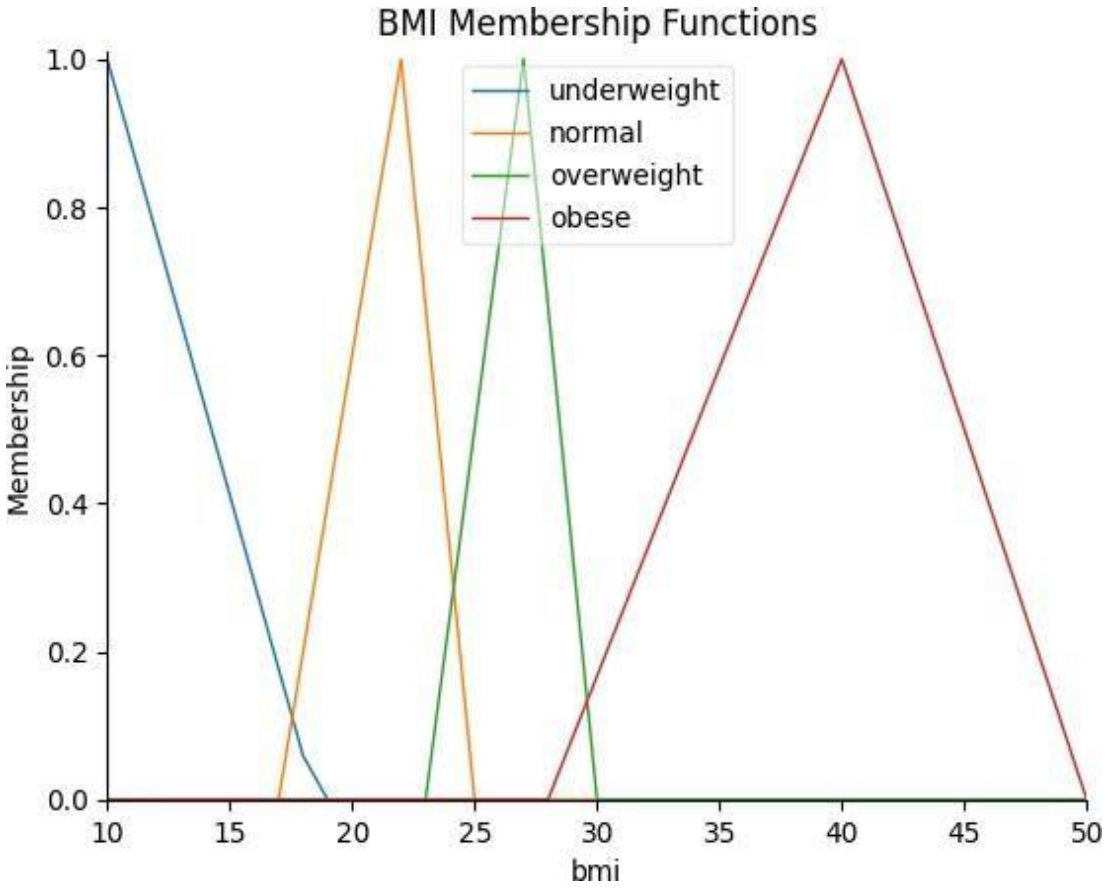

# ============================================================
# Step 17: Display Defuzzification Result
# ============================================================

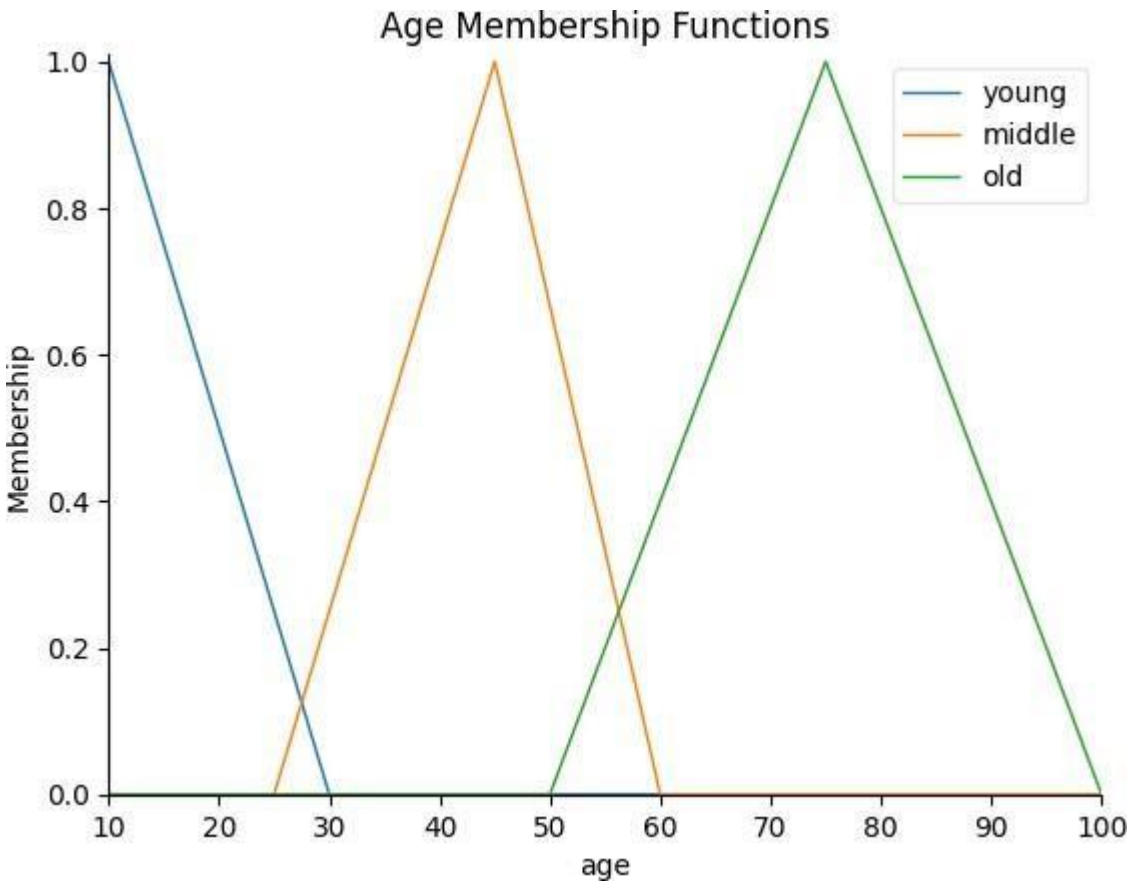
diabetes_risk.view(
    sim=diabetes_simulation
)

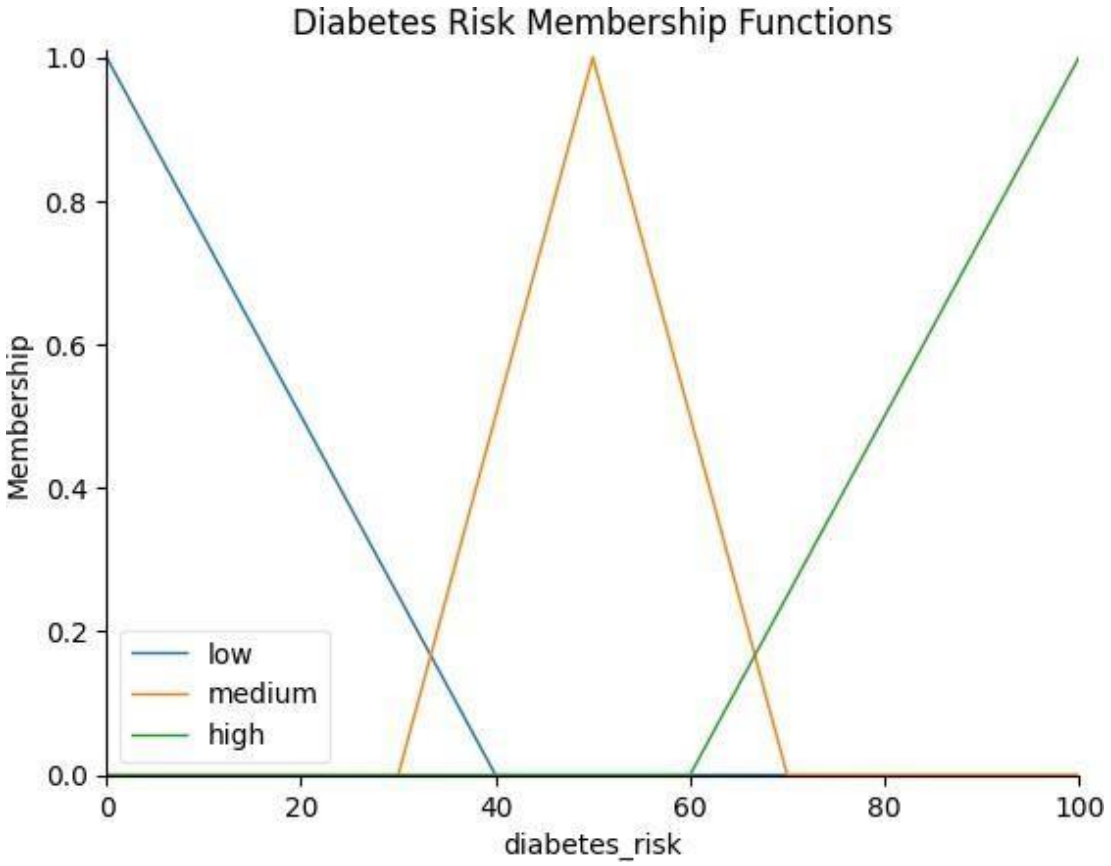
plt.title("Defuzzified Diabetes Risk")
plt.show()
```

OUTPUT:-


Blood Sugar Membership Functions
Membership
blood_sugar
low
normal
high
very_high
1.0
0.8
0.6
0.4
0.2
0.0
50
100
150
200
250
300


BMI Membership Functions
Membership
bmi
underweight
normal
overweight
obese
1.0
0.8
0.6
0.4
0.2
0.0
10
15
20
25
30
35
40
45
50


Age Membership Functions
young
middle
old
Membership
1.0
0.8
0.6
0.4
0.2
0.0
10
20
30
40
50
60
70
80
90
100
age


Diabetes Risk Membership Functions
low
medium
high
Membership
1.0
0.8
0.6
0.4
0.2
0.0
0
20
40
60
80
100
diabetes_risk

Enter blood sugar level (mg/dL): 96
Enter BMI: 35
Enter age (years): 20

==========

FUZZY DIABETES RISK SYSTEM

==========

Blood Sugar : 96.00 mg/dL
BMI : 35.00
Age : 20.00 years
Fuzzy Risk Score : 50.00 %

==========

Fuzzy Risk Level : MEDIUM

==========

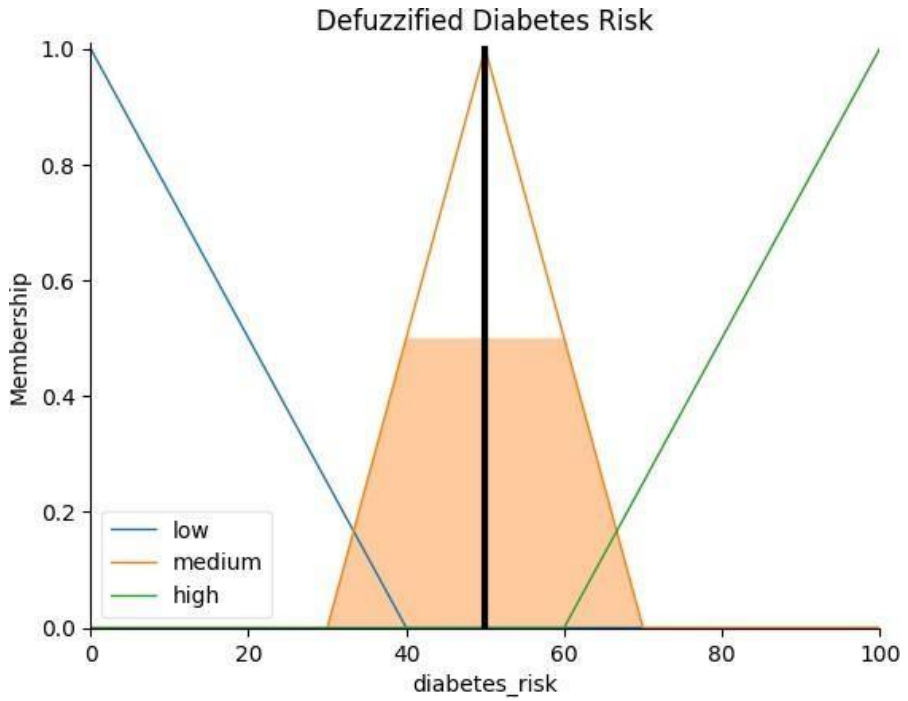
CRISP VS FUZZY DECISION

==========

Crisp Decision : LOW
Fuzzy Decision : MEDIUM
Fuzzy Risk Score : 50.00 %

==========

Result: Crisp and Fuzzy decisions are DIFFERENT.
Fuzzy logic considers Blood Sugar, BMI and Age together using gradual membership values.



PRACTICAL :- 9

Practical-9

Problem Statement

Develop a soft computing model that classifies waste into Dry or Wet categories using measurable features such as moisture level, weight, and texture score. The experiment combines a data-driven Neural Network with a rule-
based Fuzzy Logic system and evaluates the performance of the resulting hybrid intelligent system.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix



# ------------------------------------------------------------------
# 1. Sample labeled dataset
# ------------------------------------------------------------------

data = {
    "Moisture": [
        10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85,
90,
        12, 18, 28, 33, 42, 48, 58, 68, 78, 88, 22, 38, 72
    ],

    "Weight": [
        100, 150, 200, 120, 180, 220, 250, 300, 350, 400, 450, 500,
550,
        600, 650, 700, 750, 130, 170, 140, 230, 280, 320, 380, 520,
620,
        720, 190, 260, 580
    ],
```

```
    "Texture": [
        8.5, 9.0, 8.0, 7.5, 8.2, 7.0, 6.8, 6.0, 5.5, 5.0, 4.5, 4.0,
3.8,
        3.5, 3.0, 2.5, 2.0, 9.2, 8.8, 8.0, 7.2, 6.5, 6.0, 5.2, 4.2,
        3.5, 2.2, 8.3, 6.7, 3.8
    ],

    # 0 = Dry, 1 = Wet
    "Label": [
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        1, 1, 1, 1, 1, 1, 1,
        0, 0, 0, 0, 0, 0,
        1, 1, 1, 1,
        0, 0, 1
    ]
}

df = pd.DataFrame(data)


# ----------------------------------------------------------
# 2. Train/test split
# ----------------------------------------------------------

X = df[["Moisture", "Weight", "Texture"]]
y = df["Label"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

# Feature scaling
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# ------------------------------------------------------------
# 3. Neural Network
# ------------------------------------------------------------

model = MLPClassifier(
    hidden_layer_sizes=(10, 8),
    activation="relu",
    solver="adam",
    max_iter=2000,
    random_state=42
)

model.fit(X_train_scaled, y_train)

nn_prediction = model.predict(X_test_scaled)

# Probability of Wet class
nn_probability = model.predict_proba(X_test_scaled)[:, 1]


# ------------------------------------------------------------
# 4. Fuzzy membership functions
# ------------------------------------------------------------

def moisture_membership(moisture):

    if moisture <= 20:
        dry = 1.0
        wet = 0.0

    elif moisture < 60:
        dry = (60 - moisture) / 40
        wet = (moisture - 20) / 40

    else:
        dry = 0.0
        wet = 1.0

    return dry, wet


def texture_membership(texture):
```

```
    if texture <= 3:
        rough = 1.0
        dry_texture = 0.0

    elif texture < 7:

        rough = (7 - texture) / 4
        dry_texture = (texture - 3) / 4

    else:
        rough = 0.0
        dry_texture = 1.0

    return rough, dry_texture


def fuzzy_classifier(moisture, weight, texture):

    dry_moisture, wet_moisture = moisture_membership(moisture)

    rough_texture, dry_texture = texture_membership(texture)

    # ------------------------------------------------------------
    # Fuzzy Rules
    # ------------------------------------------------------------

    wet_rule_1 = wet_moisture
    wet_rule_2 = min(wet_moisture, rough_texture)

    dry_rule_1 = dry_moisture
    dry_rule_2 = min(dry_moisture, dry_texture)

    # Combine rules
    wet_score = max(wet_rule_1, wet_rule_2)
    dry_score = max(dry_rule_1, dry_rule_2)

    # Final fuzzy class
    if wet_score > dry_score:
        fuzzy_class = 1
    else:
        fuzzy_class = 0
```

```
    # Convert scores into probability
    total = wet_score + dry_score

    if total != 0:
        fuzzy_wet_probability = wet_score / total
    else:
        fuzzy_wet_probability = 0.5
    return fuzzy_wet_probability, fuzzy_class


# ------------------------------------------------------------
# 5. Hybrid Model
# ------------------------------------------------------------

hybrid_predictions = []

for i in range(len(X_test)):

    moisture = X_test.iloc[i]["Moisture"]
    weight = X_test.iloc[i]["Weight"]
    texture = X_test.iloc[i]["Texture"]

    fuzzy_probability, _ = fuzzy_classifier(
        moisture,
        weight,
        texture
    )

    # 60% Neural Network
    # 40% Fuzzy Logic

    hybrid_probability = (
        0.60 * nn_probability[i] +
        0.40 * fuzzy_probability
    )

    final_prediction = 1 if hybrid_probability >= 0.50 else 0

    hybrid_predictions.append(final_prediction)


# ------------------------------------------------------------
```

```
# 6. Evaluation
# ------------------------------------------------------

accuracy = accuracy_score(
    y_test,
    hybrid_predictions
)
print("HYBRID MODEL ACCURACY:",
      round(accuracy * 100, 2), "%")


print("\nCLASSIFICATION REPORT:")

print(
    classification_report(
        y_test,
        hybrid_predictions,
        target_names=["Dry", "Wet"]
    )
)


cm = confusion_matrix(
    y_test,
    hybrid_predictions
)

print("\nCONFUSION MATRIX:")
print(cm)


# ------------------------------------------------------------
# 7. Classify a new waste item
# ------------------------------------------------------------

new_moisture = 72
new_weight = 450
new_texture = 3.5

new_data = np.array([
    [new_moisture, new_weight, new_texture]
])
```

```
# Scale new data
new_scaled = scaler.transform(new_data)

# Neural Network probability
new_nn_probability = model.predict_proba(
    new_scaled
)[0][1]

# Fuzzy probability
new_fuzzy_probability, _ = fuzzy_classifier(
    new_moisture,
    new_weight,
    new_texture
)

# Hybrid probability
new_hybrid_probability = (
    0.60 * new_nn_probability +
    0.40 * new_fuzzy_probability
)

# Final classification
final_class = (
    "Wet"
    if new_hybrid_probability >= 0.50
    else "Dry"
)


print("\nNEW WASTE")

print("Moisture:", new_moisture, "%")
print("Weight:", new_weight, "g")
print("Texture:", new_texture)

print(
    "NN Wet Probability:",
    round(new_nn_probability, 3)
)

print(
    "Fuzzy Wet Probability:",
```

```
    round(new_fuzzy_probability, 3)
)

print(
    "Hybrid Wet Probability:",
    round(new_hybrid_probability, 3)
)
print("FINAL CLASS:", final_class)


# ----------------------------------------------------------
# 8. Confusion Matrix Plot
# ----------------------------------------------------------

plt.figure(figsize=(6, 5))

plt.imshow(
    cm,
    cmap="Blues"
)

plt.title("Hybrid Model Confusion Matrix")

plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")

plt.xticks(
    [0, 1],
    ["Dry", "Wet"]
)

plt.yticks(
    [0, 1],
    ["Dry", "Wet"]
)


# Display values inside matrix
for i in range(2):
    for j in range(2):
        plt.text(
            j,
```

```
                i,
                cm[i, j],
                ha="center",
                va="center"
            )

plt.colorbar()
plt.tight_layout()

plt.show()
```

OUTPUT:-

HYBRID MODEL ACCURACY: 100.0 %

CLASSIFICATION REPORT:

	precision	recall	f1-score	support
Dry	1.00	1.00	1.00	5
Wet	1.00	1.00	1.00	3
accuracy			1.00	8
macro avg	1.00	1.00	1.00	8
weighted avg	1.00	1.00	1.00	8

CONFUSION MATRIX:

[[5 0]
[0 3]]

NEW WASTE

Moisture: 72 %

Weight: 450 g

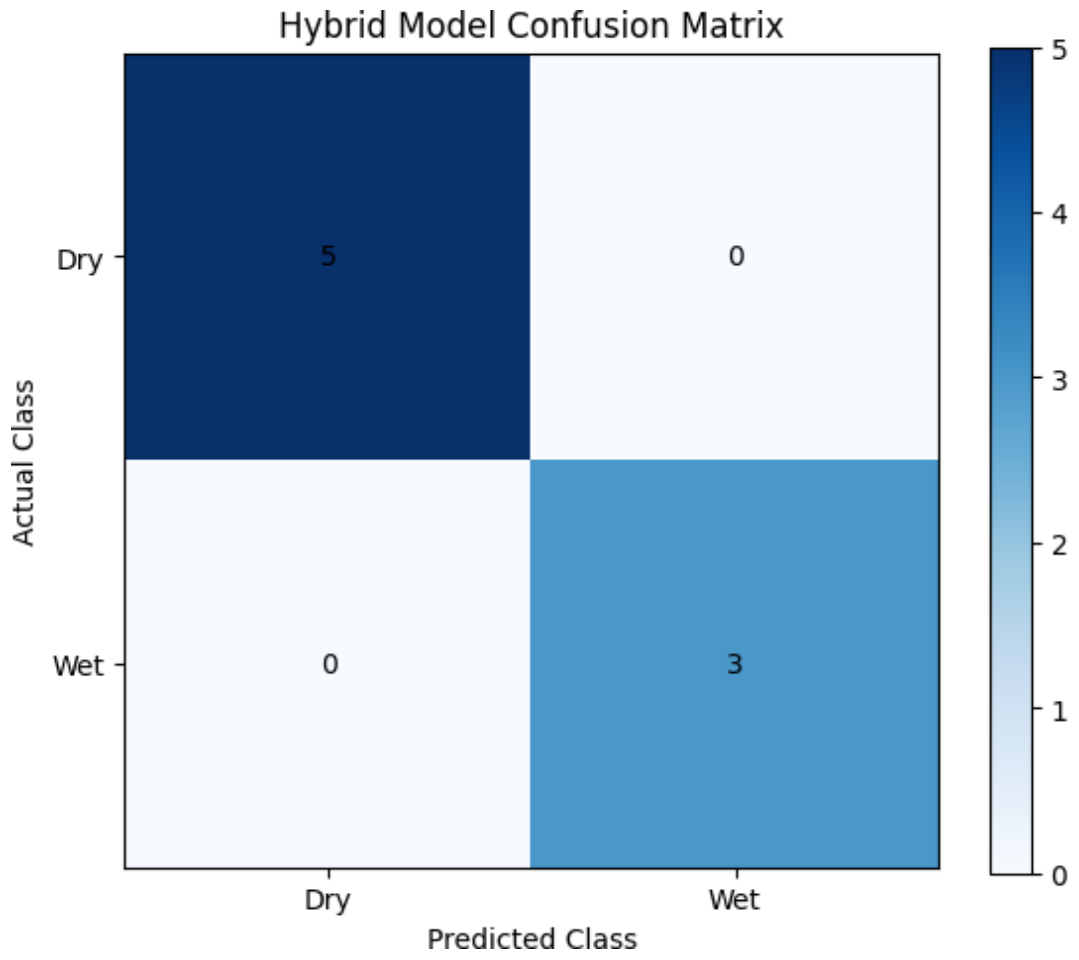
Texture: 3.5

NN Wet Probability: 0.973

Fuzzy Wet Probability: 1.0

Hybrid Wet Probability: 0.984

FINAL CLASS: Wet


Hybrid Model Confusion Matrix
Dry
Wet
Actual Class
5
0
0
3
Dry
Wet
Predicted Class
5
4
3
2
1
0